Certified Algorithms for Combinatorial Optimization Problems

Martin Sokolov

June 2, 2025

Abstract

The Feedback Vertex Set (FVS) problem is a classical NP-complete problem that asks for the smallest set of vertices whose removal eliminates all cycles from a graph. In this thesis, we study the problem on *planar graphs*, focusing on a variant we call Feedback Vertex Set with Bounded Cycle Length (FVS-BCL), where the goal is to intersect all cycles of a fixed length. While FVS-BCL is inapproximable within any constant factor in general graphs, the planar restriction opens up opportunities for efficient approximation.

In the first part of this work, we develop Efficient Polynomial-Time Approximation Schemes (EPTAS) for both the FVS-4CL (which targets 4-cycles) and the more general FVS-BCL problem. These schemes are achieved by adapting *Baker's technique*, a well-known method for designing approximation algorithms on planar graphs.

Next, we show that FVS-4CL is *fixed-parameter tractable* (FPT) by constructing a dynamic programming algorithm on *nice tree decompositions*, exploiting the fact that planar graphs have bounded treewidth. We also propose a strategy to extend the dynamic programming approach in order to prove the tractability of FVS-BCL. In addition, we utilize *Monadic Second Order Logic* (MSOL) to express and solve FVS-BCL instances on graphs of bounded treewidth, thereby demonstrating that the problem is fixed-parameter tractable when parameterized by the treewidth of the input graph.

Finally, we initiate the study of *certified algorithms* for FVS variants. Leveraging local search techniques and structural properties of FVS-BCL in planar graphs with polynomially bounded weights, we prove that it is possible to obtain $(1+\epsilon)$-certified solutions.

Contents

1 Introduction

The Feedback Vertex Set (FVS) problem is central to the field of computer science. In Karp [32] it is shown that there is a reduction from Satisfiability to twenty-one problems, one of which is FVS. Because of the hardness of the problem, extensive research has been conducted in order to find good approximation solutions [3, 5, 6, 16, 28]. In our paper, we focus on the Feedback Vertex Set with Bounded Cycle Length problem (FVS-BCL). The goal is to find a set of vertices that intersects every cycle of a given length.

Another restriction that is imposed is on the class of graphs, namely we are only considering planar graphs. Two prominent approaches have led to PTASs for a variety of problems on planar and H-minor-free graphs. Baker's technique [4] provides approximation algorithms for NP-Complete problems on planar graphs, while Demaine and Hajiaghayi [23] extend this idea using recursive separators to apply to several broader classes of graphs. A well-known limitation of the shifting technique is its inability to handle non-local problems, such as the FVS problem. In our paper, we impose restrictions on the problems in order to introduce a local property that allows us to devise an EPTAS for the weighted FVS-BCL.

The FVS problem has applications in many different fields. Some of them include combinatorial circuit design [31], deadlock prevention in computer systems [37, 44], VLSI chip design [29, 35], synchronous systems [38] and in artificial intelligence. More specifically, in the field of AI, the FVS problem is important for Bayesian inference [5, 6]. In the context of Bayesian inference, researchers often pursue approximation algorithms for the FVS problem, as approximations provide a practical and effective approach. A survey on the FVS problem and its applications by Festa et al. [27] gives more details on everything mentioned so far. The same survey argues that detecting cycles is an expensive operation, but ends its argument with:

> The design of efficient local search procedures is also considered a key component in developing effective computational methods for solving the FVS problem.

Building on this perspective, recent developments have focused on certified algorithms for combinatorial optimization problems [1, 14, 40]. In order to understand certified algorithms we first introduce perturbation resilience [7] - a condition that captures instances whose optimal solutions remain unchanged under small modifications to the input. This notion serves as a powerful foundation for designing local techniques that are both effective in practice and provably reliable.

Related Work

The problem is APX-complete in general graphs and the best approximation ratio so far is two and was first found by Becker and Geiger [6]. Around the same time, Bafna et al. [3] devised the first local ratio technique in order to arrive at the same approximation ratio. These algorithms were translated in primal-dual terms by Chudak et al. [16] who constructed a third algorithm which yields a 2-approximation both for the unweighted and weighted case of the FVS problem.

There has been much research on the complexity of algorithms for FVS in planar graphs. Kleinberg and Kumar [33] give the first PTAS for FVS, while Demaine and Hajiaghayi [23] give an EPTAS in the unweighted case. Le and Zheng [36] use a local search heuristic to obtain a PTAS for the unweighted FVS problem. For the weighted case, Cohen et al. [17] give a PTAS for this problem in bounded-genus graphs. An interesting open problem is giving an efficient PTAS for the weighted case of FVS in planar graphs.

On the subject of Beyond the Worst-Case Analysis, we refer to Makarychev and Makarychev [40]. In that paper certified algorithms are first described in an attempt to bridge the gap between the worst-case and structured instances. Angelidakis et al. [1] explore the notion of stability and initiate the study of certified algorithms for the Maximum Independent Set problem (MIS). They provide robust and certified algorithms for $(1 + \epsilon)$-stable instances of planar MIS, for any fixed $\epsilon > 0$. Covering and Dominating in planar graphs was further explored by Bumpus et al. [14]. They provide $(1 + \epsilon)$-certified algorithms for Dominating Set and H-Subgraph-Free-Deletion, which for any ϵ run in time $f(1/\epsilon) \cdot n^{O(1)}$ on minor-closed classes of graphs of bounded local treewidth.

Our Contributions

Our contributions can be grouped into three main categories.

First, we prove that both the unweighted and weighted variants of the FVS-4CL (which aims to break all cycles of length four) and the more general FVS-BCL problem admit efficient PTASs. This result is achieved through the application of Baker's technique.

Second, we present a dynamic programming algorithm for solving FVS-4CL on graphs with bounded treewidth.

Finally, we initiate the study of certified algorithms for both FVS-4CL and FVS-BCL. In the setting of planar graphs with polynomially bounded weights, we design a $(1 + \epsilon)$-certified algorithm that runs efficiently.

Organization of Material

In Section 2 we present every definition that is used throughout the thesis. In Section 3 we present the proof that both the unweighted and weighted FVS-4CL and FVS-BCL admit an efficient PTAS. In order to prove that we use a result from Section 4, namely that there is an efficient algorithm for solving these problems in graphs of bounded treewidth. The focus is shifted to Beyond the Worst-Case analysis in Section 5. We present the necessary lemmas to show that our problems have the structural properties, so we can devise a certified algorithm for them. In Section 6 we provide that algorithm and argue for the running time that is incurred.

2 Preliminaries

In this section, we provide the reader with the necessary information in order to follow the paper. We start with a reminder of Graph Theory. After that, we move to more advanced definitions in the field of Combinatorial Optimization. Finally, we get on topic by introducing perturbation resilience and certified algorithms from the field of Beyond the Worst-Case Analysis of Algorithms.

2.1 Graph Theory

In this section, the authors refer to Bondy and Murty [11]. A *graph* G is an ordered pair denoted as $(V(G), E(G))$. The set $V(G)$ is called a set of *vertices* and the set $E(G)$ is a set of *edges*. The incidence function ψ_G associates each edge with an unordered pair of (not necessarily unique) vertices. The ordered pair and the incidence function define the graph G. We are going to use n as the cardinality of the set V and m for the set of edges E. With each edge e of G, let there be associated a real number $w(e)$, called its *weight*. Then G, together with the weighting function $w : E \to \mathbb{R}$, is called a *weighted graph*, and denoted (G, w).

A graph can be *embedded in the plane*, or is *planar*, if it can be drawn on the plane in a way that edges intersect each other only at their ends. Such a drawing is called a *planar embedding* of the graph. A graph G is called *outerplanar* if it has a planar embedding $\widetilde{G}$ in which all the vertices lie on the boundary of the outer face. This is also called a *1-outerplanar graph*. Two forbidden minors of that class are the K_4 and $K_{2,3}$ graphs. In the next subsection, we familiarize the reader with Baker's approach [4]. Here, we define an *ℓ-outerplanar* to be a graph that has a planar embedding in which the vertices belong to at most ℓ concentric layers. Every planar graph is *ℓ-outerplanar* for some ℓ. Given a graph G, we look for the *ℓ-outerplanar* embedding of G for which ℓ is minimal.

Let $G = (V, E)$ be a graph and let $v \in V$. The *closed m-neighbourhood* of v, denoted by $N_G^m[v]$, is the set of all vertices in G that are at distance at most r from v:

$$N_G^m[v] = \{u \in V \mid \text{dist}_G(u, v) \leq m\}.$$

2.2 Combinatorial Optimization

In this section, we refer to the standard textbook on combinatorial optimization by Korte and Vygen [34] and Williamson and Shmoys [47]. Some definitions can be traced to the textbook on parameterized complexity by Cygan et al. [20].

2.2.1 NP-Complete Problems: Definitions and Variants

Two problems are of interest, namely the Feedback Vertex Set and the r-Dominating Set. The latter is a generalization of the Dominating Set problem. All of those problems belong to the class of **NP-complete** problems. The decision variant of the FVS problem can be defined as:

Definition 2.1: Feedback Vertex Set (FVS)

Input: A graph G and an integer k.
Question: Does there exist a set X of at most k vertices of G such that $G - X$ is a forest?

The weighted optimization version of the problem can be defined as:

Definition 2.2: Minimum Weight Feedback Vertex Set (MWFVS)

Instance:
- An undirected weighted graph (G, w), where $G = (V, E)$ and $w : V \to \mathbb{R}$.

Task: Find a vertex set $X \subseteq V$ such that:
- $G \setminus X$ is acyclic (i.e., a forest)
- The sum of the weights of the vertices in X is minimized.

The suffix 'BCL' in the definitions of the problems FVS-BCL and MWFVS-BCL indicates that they pertain to the bounded cycle length variant.

Definition 2.3: FVS-BCL

Input: A graph G, integer k and an integer ρ.
Question: Does there exist a set X of at most k vertices such that every cycle of length ρ contains a vertex in X?

The following is the weighted version of the problem; we define it as an optimization task.

Definition 2.4: MWFVS-BCL

Instance:
- An undirected weighted graph (G, w), where $G = (V, E)$ and $w : V \to \mathbb{R}$ and an integer ρ.

Task: Find a vertex set $X \subseteq V$ such that:
- Every cycle in $G \setminus X$ of length ρ has a vertex in X.
- The sum of the weights of the vertices in X is minimized.

r-Dominating Set is a generalization of Dominating Set. Setting the parameter r to one would transform the former problem into the latter.

Definition 2.5: The r-Dominating Set Problem.

Input: A graph $G = (V, E)$, an integer k and an integer r.
Question: Does there exist a subset $D \subseteq V$ of size at most k, such that every vertex $v \in V$ is within distance at most r from some vertex in D; that is,

$$\forall v \in V,\ \exists u \in D \text{ such that } \operatorname{dist}_G(u, v) \leq r.$$

The weighted version of the problem is defined as follows:

Definition 2.6: Weighted r-Dominating Set

Instance:

- An undirected graph $G = (V, E)$.
- A weight function $w : V \to \mathbb{N}$.
- An integer radius r

Task: Find a vertex set $D \subseteq V$ such that:

- Every vertex $v \in V$ is within distance at most r from some vertex in D; that is, $\forall v \in V$, $\exists u \in D$ such that $\text{dist}_G(u, v) \leq r$.
- The total weight of the vertices in D is minimized.

2.2.2 Approximation Algorithms and Complexity Classes

In the previous section all of the problems were NP-complete. In order to cope with the hardness of those problems, one can turn to *approximation algorithms*. Let Π be an optimization problem with nonnegative weights and $q \geq 1$. A *q-factor approximation algorithm* for Π is a polynomial-time algorithm A for Π, such that

$$\frac{1}{q} \cdot OPT(I) \leq A(I) \leq q \cdot OPT(I)$$

for all instances I of Π. The first inequality shown is true for maximization problems, while the second one holds for minimization problems. We say A has a performance ratio of q. If a problem falls into that category, we say that it is APX-COMPLETE.

The previous paragraph introduced the class of problems APX in the context of approximation algorithms. Now, we introduce another important class: PTAS, which consists of algorithms that run in polynomial time and produce solutions arbitrarily close to optimal, depending on a trade-off parameter.

Definition 2.7: Polynomial-Time Approximation Scheme (PTAS)

A polynomial-time approximation scheme (PTAS) is a family of algorithms A_ϵ where there is an algorithm for each $\epsilon > 0$, such that A_ϵ is a $(1 + \epsilon)$-approximation algorithm (for minimization problems). The running time of the algorithm A_ϵ is polynomial, but can depend arbitrarily on $1/\epsilon$.

The dependence, mentioned in Definition 2.7, on $\frac{1}{\epsilon}$ could be exponential or even worse. Thus an algorithm running in time $O(n^{1/\epsilon})$ counts as a PTAS.

Every problem with a PTAS is also in APX. The relation between these two classes is similar to the one between P and NP. For more on the topic, such as APX-completeness, L-reductions and the PCP-theorem, we refer to Korte and Vygen [34].

An important distinction in the complexity zoo is made in Cesati and Trevisan [15]. To the best of the knowledge of the author, Cesati and Trevisan first describe what an efficient PTAS is.

Definition 2.8: Efficient Polynomial-Time Approximation Scheme (EPTAS)

An efficient PTAS yields a $(1 + \epsilon)$ approximation scheme and has a running time of $f(\epsilon) \cdot n^c$ where f is an arbitrary function and c is a constant.

2.2.3 Baker's approach

This subsection presents the shifting technique, originally introduced in Baker's seminal paper [4]. The technique outlines a general algorithm that partitions the graph into a disjoint union of vertex levels. By analysing slices of these levels and, more specifically, the overlap between these slices, one obtains an approximation ratio for the solution to the optimization problem. The slices make up several levels that are specifically chosen for a problem and it should hold that an exact solution can be found in each one slice. It must also hold that this solution can be found in optimal time.

There is also another way to view this approach. Instead of analysing the overlap between slices, one could partition the graph by removing said overlap. In this way, the resulting disjoint union may not cover the entire optimal solution, but it can be shown that the deviation from optimality is bounded. This approach works for hereditary maximization problems like Independent Set, Max-Cut, Maximum P-Matching, etc.

The following section presents one example application. It focuses on the general shifting technique, using the Vertex Cover problem as an illustrative case. This example is explored in great detail, as both the problem and the technique closely resemble the approach required for solving FVS-BCL.

It is important to highlight that the general shifting technique is more powerful than the vertex-removal partitioning. For problems like Dominating Set, the latter technique does not yield a PTAS.

However, using the general shifting technique, one can obtain a PTAS for Dominating Set in planar graphs. This was done by Marzban and Gu [41]. They designed a PTAS with an approximation ratio of $(1 + 2/\ell)$. The two in the approximation ratio appears because of the overlap on two levels.

What follows is a brief explanation of Baker's shifting technique with Vertex Cover as an example.

Algorithm 1 Baker's shifting technique

Require: Planar graph $G = (V, E)$, parameter ℓ

Ensure: $(1 + \epsilon)$-approximation for Vertex Cover

1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j + 1) \cdot \ell + i$ for $0 \leq i < \ell$.
5: $\forall j$ compute $S_{i,j}$, the min. VC of $G_{i,j}$.
6: $\forall j$ let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup S_i$
8: **end for**
9: **return** S_{i*} from S with minimum cardinality

Analysis of the algorithm:

Let V_i be the set of vertices at levels $i \mod \ell$:

$$V_i = \{v \in V \mid \text{level}(v) \equiv i \mod \ell\}$$

Let OPT_i denote $OPT \cap V_i$. We notice that OPT_i are disjoint and $OPT = \bigcup_i OPT_i$. Therefore, we can make the following argument:

1. $\exists q : |OPT_q| \leq \frac{1}{\ell} \cdot OPT = \epsilon |OPT|$
2. $S_q \leq \sum_j S_{q,j}$, for some shift q we have this inequality because in step 5. of the algorithm we define the set to be a union over all slices and some vertices may appear twice in the boundary layers.
3. $\sum_j S_{qj} \leq \sum_j |OPT \cap V(G_{q,j})|$, we know that $S_{q,j}$ is a min. VC for $G_{q,j}$, while the optimal solution may choose different vertices, especially on the boundary for the global optimality of the solution.
4. $\sum_j |OPT \cap V(G_{q,j})| \leq |OPT| + |OPT_q|$, if we consider the graphs $\{G_{q,j}\}$, then we can argue that each vertex appears once, except those at levels $q \mod \ell$, which appear twice.

From the first, second and fourth step we can conclude that:

$$S_q \leq |OPT| + |OPT_q| \leq (1 + \epsilon) \cdot |OPT|$$

2.2.4 Treewidth

According to Diestel [25, pp. 354-355] the notions of tree decomposition and treewidth were introduced by Halin [30] under the name of S-functions and Robertson and Seymour [42]. We give brief definitions of these concepts and then discuss bounded local treewidth and linear local treewidth. The last two notions are important for the main theorem we work with, namely Bumpus et al. [14, Theorem 1.7].

From Diestel [25, Section 12.3] we have the following definition of a tree decomposition.

Definition 2.9: Tree Decomoposition

Let G be a graph, T a tree, and let $\mathcal{V} = (V_t)_{t \in T}$ be a family of vertex sets $V_t \subseteq V(G)$ indexed by the vertices t of T. The pair $(T, \mathcal{V})$ is called a *tree decomposition* of G if it satisfies the following conditions:
(T1) $V(G) = \bigcup_{t \in T} V_t$.
(T2) For every edge $e \in E(G)$, there exists a $t \in T$ such that both ends of e lie in V_t.
(T3) If $t_1, t_2, t_3 \in T$ satisfy t_2 lies on the unique path between t_1 and t_3 in T, then

$$V_{t_1} \cap V_{t_3} \subseteq V_{t_2}.$$

The *width* of a tree decomposition (T, V) is the size of its largest set V_t minus one. The *treewidth* $\text{tw}(G)$ of a graph G is the minimum width among all possible tree decompositions of G.

Definition 2.10: Bounded local treewidth

A graph G has bounded local treewidth if the subgraph induced by vertices at distance at most d is a function of d ($f : \mathbb{N} \to \mathbb{R}$) and not n.

Furthermore, if there exists $\lambda \in \mathbb{R}$, such that f can be defined as $f : r \to \lambda \cdot r$, then G would have linear local treewidth.

We know that planar graphs are a class of graphs that also have linear local treewidth [8, 26, 24].

2.2.5 Linear Programming and Randomized Rounding

One way of approximating solutions is done through Linear Programming (LP). For a more detailed review of LP, the reader can consult Korte and Vygen [34, Chapter 3]. One solves a linear relaxation, which provides fractional values as solutions. Afterwards, the fractional solution is rounded to an integral one, but rather than doing it deterministically, this process is done in a randomized fashion. This technique is called *randomized rounding*. This framework provides a way to obtain certified algorithms, for a more in depth look for randomized rounding the reader should consult Makarychev and Makarychev [40].

2.3 Beyond the Worst-Case Analysis of Algorithms

In this section we explore perturbation-resilience, also known as γ-*stability* or Bilu-Linial stability. The authors refer to the original paper of Bilu and Linial [7] as well as a survey by Makarychev and Makarychev [39]. In a later work, Makarychev and Makarychev define γ-*certified algorithms* [40].

The following definitions for γ-perturbation are for edge-optimization and vertex-optimization problems, respectively. Subsequently, the notions of stability and certified algorithms are introduced specifically for vertex-optimization problems.

Definition 2.11: γ-perturbation for Edge-Optimization problems

Consider a weighted graph $G = (V, E, w)$ with vertex set V, edge set E, and a weight function w defined on the edges. An instance $G' = (V, E, w')$ is a γ-*perturbation* (for $\gamma \in \mathbb{R}_{\geq 1}$) of G if $w(e) \leq w'(e) \leq \gamma \cdot w(e)$ for every edge $e \in E$; in other words, the perturbed instance is obtained by independently scaling the weight of each edge by a factor between 1 and γ.

From now on, the definitions will concentrate on Vertex-Optimization problems.

Definition 2.12: γ-perturbation for Vertex-Optimization problems

Consider a weighted graph (G, w). For any $\gamma \in \mathbb{R}_{\geq 1}$ a γ-perturbation of the weight function $w : V \to \mathbb{R}$ is a function $w' : V \to R$ satisfying $w(v) \leq w'(v) \leq \gamma \cdot w(v)\ \forall v \in V$.

Definition 2.13: γ-stability

For any $\gamma \in \mathbb{R}_{\geq 1}$, a γ-stable instance (G, w) of a *vertex-minimization* problem Π is an instance admitting a unique optimal solution S which remains optimal even under γ-perturbations of (G, w).

Definition 2.14: Certified Algorithm

A γ-certified solution to an instance (G, w) of a weighted vertex-optimization problem Π is a pair (S, w') where w' is a γ-perturbation of w and S is an optimal solution on (G, w'). A γ-certified algorithm for Π maps instances of Π to γ-certified solutions.

2.4 r-Dominating Set; Exact algorithm and PTAS

For FVS-BCL we will give an algorithm that solves the problem on graphs of bounded treewidth. For Dominating Set and its generalizations there are already existing solutions one can use. The r-Dominating

Set problem is one generalization of Dominating Set. We mention an algorithm which solves the generalized version. In this work the parameter r is considered to be constant.

There is an algorithm that solves r-Dominating Set using dynamic programming which is realized by Borradaile and Le [13, Theorem 1, Appendix B.]. It is a generalization of the work done in van Rooij et al. [45]. As it is argued in Borradaile and Le [13, Appendix B.], the running time needed to solve r-Dominating Set is $(2r+1)^{\mathrm{tw}+1} \cdot \mathrm{tw}^2 \cdot n$.

We showed in subsection 2.2.3 how one can use an exact algorithm for graphs of bounded treewidth to get an approximation in a planar graph. In Borradaile and Le [13] it is mentioned that one can use Baker's technique to obtain a PTAS. In the MIT lecture notes of [22], there is an explanation of how to get to that approximation. Essentially, instead of two layers of overlap between subgraphs, one would have to increase that overlap to $r+1$. This technique yields results for constant values of r.

3 EPTAS for FVS-4CL and FVS-BCL using Baker's technique

This section presents an EPTAS for both the unweighted and weighted versions of the FVS-BCL problem. We begin with the unweighted case, focusing on the algorithm's correctness and efficiency.

For the special case of breaking 4-cycles—denoted as FVS-4CL—exact solutions on subgraphs of bounded treewidth are obtained using a dynamic programming approach. In contrast, the general FVS-BCL problem, where cycles of arbitrary length ρ must be broken, is handled using Monadic Second Order Logic (MSOL) and Courcelle's Theorem.

The running time of the overall approach remains polynomial in the size of the input.

Some notation used throughout this section is introduced here. The 4-cycle-breaking problem is denoted as FVS-4CL and is used consistently in subsections 3.1 and 3.2, regardless of whether the input is weighted. In subsection 3.3, where arbitrary cycle lengths are considered, the notation FVS-BCL is used. This distinction is maintained in subsequent chapters. Unless stated otherwise, FVS-4CL refers to the weighted variant of the 4-cycle-breaking problem.

3.1 Baker's technique for unweighted FVS-4CL on planar graphs

In this subsection we provide the necessary definitions, lemmas and theorems to demonstrate how to obtain an EPTAS for the problem we defined in 2.3, namely FVS-4CL. In the preliminaries we gave two examples of how to construct algorithms using Baker's technique and how to further analyse them. In the same fashion we now construct Algorithm 2 and reason for its effectiveness.

In the following algorithm we use the letter ℓ for the outerplanarity of each subgraph, i.e. each $G_{i,j}$ that we define on Line 4 in Algorithm 2 is ℓ-outerplanar.

Algorithm 2 Baker's technique for the unweighted FVS-4CL

Require: Planar graph $G = (V, E)$, parameter $\ell \leftarrow \frac{1}{\epsilon}$
Ensure: $(1+\epsilon)$-approximation for FVS-4CL
1: Perform BFS from some arbitrary vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j+1) \cdot \ell + i + 1$ for all $j \geq 0$.
5: For each j, compute $S_{i,j}$, the minimum unweighted FVS-4CL of $G_{i,j}$ using Algorithm 4.4 as a subroutine (weights are all ones).
6: Let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup \{S_i\}$
8: **end for**
9: **return** S_{i^*} from S with minimum cardinality

In the analysis of the algorithm we are going to need three lemmas in order to prove that Algorithm 2

obtains a $(1 + 2/\ell)$-approximation of the optimum solution.

Lemma 3.1: Bound on the Optimum Solution in Subgraphs (Unweighted Case)

Let $S_{i,j}$ be the minimum unweighted feedback vertex set (FVS-4CL) for subgraph $G_{i,j}$, computed using Algorithm 2, and let $F \equiv \text{OPT}$ be the optimum solution for the full graph G.
Define $F_{i,j} := F \cap V(G_{i,j})$, i.e., the restriction of the global solution F to the subgraph $G_{i,j}$.
Then:

$$|S_{i,j}| \leq |F_{i,j}|$$

Proof. We prove this in three steps by fixing a particular shift i and slice j, and analysing the corresponding subgraph $G_{i,j}$, which is induced on the set of vertices at levels from $j \cdot \ell + i$ to $(j + 1) \cdot \ell + i + 1$.

1. **Locality of 4-cycles:** Any 4-cycle that is fully contained in $G_{i,j}$ must be broken by vertices within $G_{i,j}$. That is, no vertex $v \in V(G) \setminus V(G_{i,j})$ can break 4-cycles entirely within $G_{i,j}$.
2. **Feasibility of restricted solution:** Since F is a valid global solution for the 4-cycle-breaking problem in G, its restriction to $G_{i,j}$, namely $F_{i,j} = F \cap V(G_{i,j})$, must also be a feasible 4-cycle-breaking set for the subgraph $G_{i,j}$.
3. **Optimality of $S_{i,j}$:** By assumption, $S_{i,j}$ is the minimum unweighted solution for $G_{i,j}$. Since $F_{i,j}$ is also feasible (though not necessarily optimal), we have:

$$|S_{i,j}| \leq |F_{i,j}|.$$

This argument holds for arbitrary values of i and j, so we conclude that:

$$\forall i, j : \quad |S_{i,j}| \leq |F_{i,j}|.$$

□

One must confront the fact notation is abused throughout the paper, especially when it comes down to j, the slice variable. When we say that a statement holds for all values i and j, we mean that it holds for the subgraphs that these variables outline. We will sketch an example to deepen our understanding of Lemma 3.1. Moreover, we are going to explore one avenue that gives rise to an inequality in the lemma.

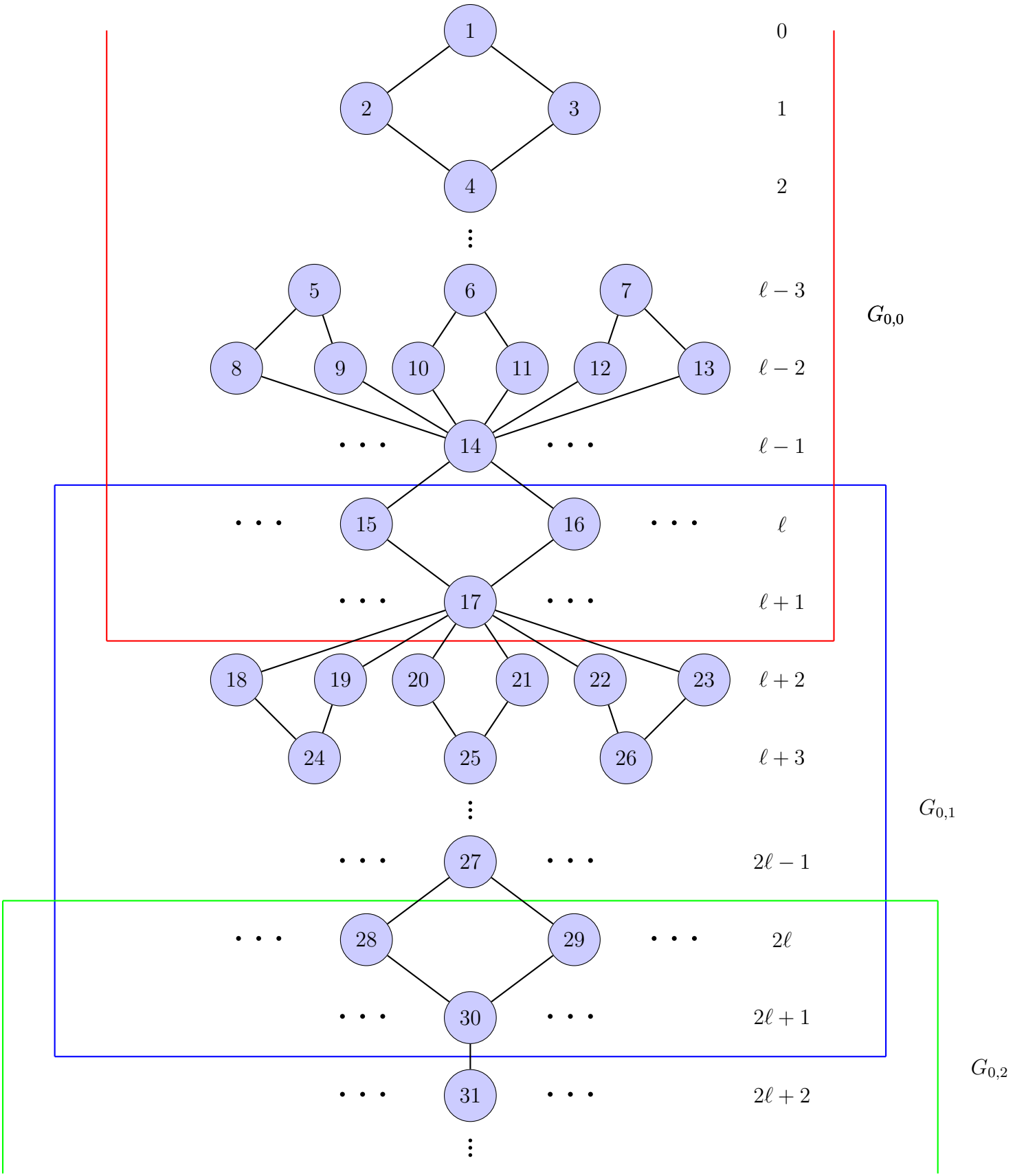

Figure 1: BFS tree, showing the distance from the root for each layer.

Let G look like the graph we sketched on the previous page. The vertical dots help us clarify the levels

while the horizontal signify that there may be other vertices of interest. However, there are vertices we would like to inspect, specifically vertices with labels 14 and 17. Let $v_{14}, v_{17} \in F$. What follows is that $v_{14}, v_{17} \in F_{0,0}$. The cycles that give us reason to claim that $v_{14} \in F$ are all within $G_{0,0}$. Therefore, it holds that Algorithm 2 will have $v_{14} \in S_{0,0}$. However, the same does not hold for v_{17}, the cycles that it is a part of are in $G_{0,1}$ and thus $v_{17} \notin S_{0,0}$, while $v_{17} \in F_{0,0}$. This example should not distract the reader from the fact the real reason that $|S_{0,0}| \leq |F_{0,0}|$ is because $S_{0,0}$ is by definition the minimum 4-cycle-breaking set for $G_{0,0}$, while $F_{0,0}$ is, as we previously argued, a feasible 4-cycle-breaking set.

What follows from Lemma 3.1 is a result that holds for any shift i.

Result 3.2: Optimum solution bound for the whole graph (Unweighted Case)

Let S_i be the union of optimal solutions defined on Line 6 of Algorithm 2 for some shift i. Let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. For those sets it holds that:

$$|S_i| \leq \sum_j |S_{i,j}| \leq \sum_j |F_{i,j}|.$$

The first inequality in Result 3.2 stems from the fact that some vertices in the $S_{i,j}$ sum may appear twice. This pertains to vertices that are located on the boundary. For example if the $v_{30} \in S_{0,1} \wedge v_{30} \in S_{0,2}$, then it would be counted once in S_0, but twice in $\sum_j S_{0,j}$. The second inequality is a result from Lemma 3.1 where we argue that $S_{i,j}$ is an optimum solution for $G_{i,j}$ while $F_{i,j}$ is a 4-cycle-breaking set, but not one of necessarily minimum cardinality.

In Lemma 3.4 we look at the bound of the union of the $F_{i,j}$ sets. Before that, however, we make an argument for the vertices on the boundaries in Lemma 3.3.

Lemma 3.3: Bound for the vertices on the boundaries (Unweighted Case)

Let $F_i = F \cap \{\text{vertices at levels } i \bmod \ell\}$. The sets F_i are disjoint and $\bigcup_i F_i = F$.
We claim that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} : |F_q| + |F_{q+1}| \leq \frac{2}{\ell} \cdot |F|.$$

Proof. Assume for the sake of contradiction that

$$\forall i \in \{0, 1, \ldots, \ell - 1\} : \quad |F_i| + |F_{i+1}| \; > \; \frac{2}{\ell} |F|. \tag{1}$$

Then we argue in three steps:

1. Summing 1 over all $i = 0, \ldots, \ell - 1$ gives

$$\sum_{i=0}^{\ell-1} \big(|F_i| + |F_{i+1}|\big) \; > \; \sum_{i=0}^{\ell-1} \frac{2}{\ell} |F| \; = \; 2\,|F|.$$

2. On the other hand, since the sets $F_0, \ldots, F_{\ell-1}$ are pairwise disjoint and $\bigcup_i F_i = F$, each F_i appears

exactly twice in the left-hand sum. Therefore

$$\sum_{i=0}^{\ell-1}\big(|F_i| + |F_{i+1}|\big) = 2\sum_{i=0}^{\ell-1}|F_i| = 2\,|F|,$$

yielding a contradiction.

3. Hence the assumption $\forall i:\ |F_i| + |F_{i+1}| > \frac{2}{\ell}|F|$ must be false. By the pigeonhole principle there is at least one index
$$q \in \{0, 1, \ldots, \ell-1\} \quad \text{such that} \quad |F_q| + |F_{q+1}| \ \leq\ \frac{2}{\ell}\,|F|.$$

□

Now we argue for the size of the sets $F_{i,j}$. It is natural to try and tie their size to the size of the optimal solution F using Lemma 3.3. Whenever we refer to F_i and F_{i+1}, one should keep in mind that the second term may wrap around and become F_0.

Lemma 3.4: Bound for the cardinality of the intersection sets (Unweighted Case)

Let $F \equiv OPT$ be the optimal solution and $F_{i,j} = F \cap G_{i,j}$ be the intersection sets with the subgraphs. Then we have:

$$\sum_j |F_{i^*,j}| \leq |F| + \frac{2}{\ell}\cdot|F|$$

for some specific integer i^*.

Proof. If we look at Fig. 1, then we notice that any vertex that is located in the overlap layers would be counted twice in the sum $\sum_j |F_{i,j}|$. The interior vertices are counted once towards the optimum solution sum. Therefore, we have that:

1. $\sum_j |F_{i,j}| = |F| + |\{\text{vertices on the boundary}\}| = |F| + |F_i| + |F_{i+1}|$ for $i \in \{0, 1, \ldots, \ell-1\}$.
2. For the RHS of the equation in step 1. we argue that:
$$\exists i^* : |F_{i^*}| + |F_{i^*+1}| \leq \frac{2}{\ell}\cdot|F|$$
using Lemma 3.3.
3. For that i^* it holds that $\sum_j |F_{i^*,j}| \leq |F| + \frac{2}{\ell}|F|$.

□

Finally, we chain the inequalities presented in Lemma 3.1, Result 3.2 and Lemma 3.4, and get that for some slice i^*, it holds that:

$$|S_{i^*}| \leq \sum_j |S_{i^*,j}| \leq \sum_j |F_{i^*,j}| \leq |F| + \frac{2}{\ell}\cdot|F| = (1 + \frac{2}{\ell})\cdot|F|$$

.

The inequality

$$|S_{i^*}| \leq \left(1 + \frac{2}{\ell}\right) \cdot |F|$$

implies that Algorithm 2 returns a $\left(1 + \frac{2}{\ell}\right)$-approximate solution. Since we set $\ell \leftarrow \frac{1}{\epsilon}$, this yields a $(1 + 2\epsilon)$-approximation for the unweighted planar FVS-4CL problem.

Running time:

Lines 1 and 9 of Algorithm 2 can be computed in linear time $O(n)$. The loop that starts on Line 3 has a running time that is dominated by step 5 where we call Algorithm 4.4 (with weights equal to ones) as a subroutine. That algorithm takes as input subgraphs that are $\ell + 2$-outerplanar. As we argued in the Preliminaries section, such graphs have bounded treewidth of tw, Bodlaender [8, Theorem 83]. Therefore, Algorithm 2 has a running time of $2^{\mathcal{O}(\text{tw}^2)} \cdot n$.

3.2 Baker's technique for the weighted FVS-4CL on planar graphs

In this subsection we look at the weighted version of the problem. Firstly, we construct Algorithm 3 which differs from Algorithm 2 only on step 5. On Line 5 we construct Algorithm 4 and use it as a subroutine to get the solution we want.

Algorithm 3 Baker's technique for the weighted FVS-4CL

Require: Planar graph $G = (V, E)$, vertex weight function $w : V \to \mathbb{R}_{\geq 0}$, parameter $\ell \leftarrow \frac{1}{\epsilon}$
Ensure: $(1 + \epsilon)$-approximation for FVS-4CL
1: Perform BFS from some arbitrary root vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j + 1) \cdot \ell + i + 1$, for all $j \geq 0$
5: For each j, solve the weighted FVS-4CL problem exactly on the subgraph $G_{i,j}$ using Algorithm 4.4 as a subroutine, and let the solution be $S_{i,j}$
6: Let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup \{S_i\}$
8: **end for**
9: **return** $S_{i^*} \in S$ such that $w(S_{i^*}) \leq w(S_i)$ for all $i \in \{0, 1, \ldots, \ell - 1\}$

Much like the proof we did in subsection 3.1, we start off with finding a bound for the optimum solution for each subgraph.

Lemma 3.5: Optimum solution bound (Weighted Case)

Let $G = (V, E)$ be a planar graph and $w : V \to \mathbb{R}$ be a weight function. Let $S_{i,j}$ be an optimum solution for $G_{i,j}$ for some shift i and slice j, let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. In the weighted case we claim that:

$$w(S_{i,j}) \leq w(F_{i,j}).$$

Proof. We prove this in three steps by fixing a particular shift i and slice j, and analysing the corresponding subgraph $G_{i,j}$, which is induced on the set of vertices at levels from $j \cdot \ell + i$ to $(j+1) \cdot \ell + i + 1$.

1. **Locality of 4-cycles:** Any 4-cycle that is fully contained in $G_{i,j}$ must be broken by vertices within $G_{i,j}$. That is, no vertex $v \in V(G) \setminus V(G_{i,j})$ can break 4-cycles entirely within $G_{i,j}$.
2. **Feasibility of restricted solution:** Since F is a valid global solution for the 4-cycle-breaking problem in G, its restriction to $G_{i,j}$, namely $F_{i,j} = F \cap G_{i,j}$, must also be a feasible 4-cycle-breaking set for the subgraph $G_{i,j}$.
3. **Optimality of $S_{i,j}$:** By assumption, $S_{i,j}$ is the minimum-weight solution for $G_{i,j}$. Since $F_{i,j}$ is also feasible (though not necessarily optimal), we have:

$$w(S_{i,j}) \leq w(F_{i,j}).$$

This argument holds for arbitrary values of i and j, so we conclude that:

$$\forall i, j: \quad w(S_{i,j}) \leq w(F_{i,j}).$$

□

Result 3.6: Optimum solution bound for the whole graph (Weighted Case)

Let S_i be the union of optimal solutions defined on Line 6 of Algorithm 3 for some shift i. Let $F \equiv OPT$ be the optimum solution in G and let $F_{i,j} = G_{i,j} \cap F$. For those sets it holds that:

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j}).$$

Proof. This result immediately follows from the fact that when we define S_i as the union of the subgraphs some vertices, specifically those on the overlapping layers, will have their weights added twice to the sum $\sum_j w(S_{i,j})$. If we inspect the LHS, however, each vertex and its weight will be counted only once towards $w(S_i)$. Therefore, it holds that $w(S_i) \leq \sum_j w(S_{i,j})$ and from Lemma 3.5 we chain the inequalities to get Result 3.6.

□

Lemma 3.7: Bound on the Weight of Boundary Vertices (Weighted Case)

Let $G = (V, E)$ be a planar graph, and let $w : V \to \mathbb{R}$ be a weight function on the vertices. Let $F \subseteq V$ be a feedback vertex set, and define the sets F_i as:

$$F_i := F \cap \{\text{vertices at BFS level } i \bmod \ell\}$$

for each $i \in \{0, 1, \ldots, \ell - 1\}$. Then, the sets F_i are disjoint, and:

$$F = \bigcup_{i=0}^{\ell-1} F_i$$

We claim that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} \text{ such that } w(F_q) + w(F_{q+1}) \leq \frac{2}{\ell} \cdot w(F)$$

Proof. Assume by way of contradiction that the following inequality holds for all $i \in \{0, 1, \ldots, \ell-1\}$ (with indices taken modulo ℓ):

$$w(F_i) + w(F_{i+1}) > \frac{2}{\ell} \cdot w(F) \tag{2}$$

1. From the definitions of the sets F_i, we know:

$$F_i \cap F_j = \emptyset \quad \text{for all } i \neq j$$

$$F = \bigcup_{i=0}^{\ell-1} F_i$$

Therefore, each vertex in F is in exactly one F_i, and:

$$\sum_{i=0}^{\ell-1} w(F_i) = w(F)$$

When summing all pairs $w(F_i) + w(F_{i+1})$, each term $w(F_i)$ appears exactly twice (once in the pair (F_{i-1}, F_i) and once in (F_i, F_{i+1})). So:

$$\sum_{i=0}^{\ell-1} \big(w(F_i) + w(F_{i+1})\big) = 2 \cdot w(F)$$

2. Now, under the assumption of inequality (2), we would have:

$$\sum_{i=0}^{\ell-1} \big(w(F_i) + w(F_{i+1})\big) > \sum_{i=0}^{\ell-1} \frac{2}{\ell} \cdot w(F) = 2 \cdot w(F)$$

which contradicts the conclusion from step 1 that this sum equals $2 \cdot w(F)$.

3. Therefore, our assumption must be false. By the contrapositive, there must exist at least one index

$q \in \{0, 1, \ldots, \ell - 1\}$ such that:

$$w(F_q) + w(F_{q+1}) \leq \frac{2}{\ell} \cdot w(F)$$

This completes the proof.

□

Lemma 3.8: Bound for the weight of the intersection sets (Weighted Case)

Let $F \equiv OPT$ be the optimal solution of the FVS-4CL problem. Let $F_{i,j} = F \cap G_{i,j}$ be the intersection sets with the subgraphs. Then we have:

$$\sum_j w(F_{i^*,j}) \leq w(F) + \frac{2}{\ell} \cdot w(F)$$

for some specific integer i^*.

Proof. The interior vertices of each subgraph have their weights counted only once towards the sum $\sum_j w(F_{i,j})$. The same does not hold for the vertices on the boundary, they are counted once in $F_{i,j}$ and once in $F_{i,j+1}$. To account for that we rewrite the sum on the LHS as:

1. $\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\})$.
2. From Lemma 3.7 we have an integer $i^* \in \{1, 2, \ldots, \ell - 1\}$, such that:

   $w(F_{i^*}) + w(F_{i^*+1}) \leq \frac{2}{\ell} \cdot w(F)$.
3. From the previous two steps we get:

$$\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\}) \leq \frac{2}{\ell} \cdot w(F)$$

□

Finally, we chain the inequalities we obtained in Lemma 3.5, Result 3.6, and Lemma 3.8 to argue that for some $i^* \in \{1, 2, \ldots, \ell - 1\}$:

$$w(S_{i^*}) \leq \sum_j w(S_{i^*,j}) \leq \sum_j w(F_{i^*,j}) \leq w(F) + \frac{2}{\ell} \cdot w(F)$$

.

Running time:

The only difference from the unweighted case comes from Line 5 of Algorithm 3. However, using Algorithm 4.4 we get that the running time of the algorithm is still $2^{\mathcal{O}(\text{tw}^2)} \cdot n$.

3.3 Baker's technique for weighted FVS-BCL of arbitrary cycle length

This subsection extends the concepts discussed in subsection 3.2 by examining the outcome of attempting to break all weighted cycles of length ρ.

Algorithm 4 Baker's technique for the weighted FVS-BCL

Require: Planar graph $G = (V, E)$, vertex weight function $w : V \to \mathbb{R}_{\geq 0}$, parameter $\ell \leftarrow \frac{1}{\epsilon}$, integer ρ specifying the maximum cycle length to break

Ensure: $(1 + \lfloor \frac{\rho}{2} \rfloor \cdot \epsilon)$-approximation for FVS-BCL

1: Perform BFS from some arbitrary root vertex r
2: $S \leftarrow \emptyset$
   i: *shift*; j: *slice*
3: **for** each $i = 0$ to $\ell - 1$ **do**
4: Let $G_{i,j}$ be the subgraph induced on vertices at levels $j \cdot \ell + i$ through $(j+1) \cdot \ell + i + \lfloor \frac{\rho}{2} \rfloor$, for all $j \geq 0$
5: For each j, solve the weighted FVS-BCL problem exactly on the subgraph $G_{i,j}$ using Corollary 4.2, and let the solution be $S_{i,j}$
6: Let $S_i = \bigcup_j S_{i,j}$
7: $S \leftarrow S \cup \{S_i\}$
8: **end for**
9: **return** $S_{i^*} \in S$ such that $w(S_{i^*}) \leq w(S_i)$ for all $i \in \{0, 1, \ldots, \ell - 1\}$

The changes between Algorithm 3 and Algorithm 4 pertain to Lines 4-7. The main difference is on Line 4 where we define the subgraphs we want to inspect. This change enables us to argue that the union of the local solutions, defined on Line 6, is indeed a feasible solution for the whole graph.

Lemma 3.9: Optimum solution bound for FVS-ρCL

Let $G = (V, E)$ be a planar graph, $w : V \to \mathbb{R}$ be a weight function and ρ an integer. Let $F \equiv OPT$ be the optimum solution in the whole graph for FVS-ρ-BCL and $F_{i,j} = G_{i,j} \cap F$. Let $S_{i,j}$ be an optimum solution for $G_{i,j}$ for some shift i and slice j. We claim that:

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j}).$$

Proof. We prove this in three steps by fixing a particular shift i and analysing the solution constructed from slices $G_{i,j}$, each induced on vertices at levels from $j \cdot \ell + i$ to $(j+1) \cdot \ell + i + \lfloor \frac{\rho}{2} \rfloor$, where ρ is the maximum cycle length we wish to break.

1. **Locality of ρ-cycles:** Any cycle of length ρ that is entirely contained within $G_{i,j}$ must be broken by vertices from within $G_{i,j}$. That is, no vertex $v \in V(G) \setminus V(G_{i,j})$ can break such cycles in $G_{i,j}$.

2. **Feasibility of restricted solution:** Since F is a valid global solution that breaks all cycles of length ρ, the restriction $F_{i,j} = F \cap V(G_{i,j})$ is a feasible solution for the ρ-cycle-breaking problem in $G_{i,j}$. By definition, $S_{i,j}$ is the optimal solution for $G_{i,j}$, so:

$$w(S_{i,j}) \leq w(F_{i,j})$$

Summing over all j, we get:

$$\sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j})$$

3. **Overlap and accounting for duplicates:** The total weight $w(S_i)$ of the combined solution across slices may be smaller than $\sum_j w(S_{i,j})$, because vertices near slice boundaries can appear in multiple $G_{i,j}$ but are only counted once in S_i. Therefore:

$$w(S_i) \leq \sum_j w(S_{i,j})$$

Chaining both inequalities, we conclude:

$$w(S_i) \leq \sum_j w(S_{i,j}) \leq \sum_j w(F_{i,j}).$$

□

Lemma 3.9: Bound on the Weight of Boundary Vertices for FVS-ρCL

Let $G = (V, E)$ be a planar graph, and let $w : V \to \mathbb{R}$ be a weight function on the vertices. Let $F \subseteq V$ be a solution to the FVS-ρCL problem (i.e., a set that breaks all cycles of length ρ). For a fixed layering from a BFS traversal, define:

$$F_i := F \cap \{\text{vertices at level } i \bmod \ell\}$$

for each $i \in \{0, 1, \ldots, \ell - 1\}$. Then the sets F_i are disjoint and together cover all of F:

$$F = \bigcup_{i=0}^{\ell-1} F_i.$$

We claim that there exists some $q \in \{0, 1, \ldots, \ell - 1\}$ such that:

$$w(F_q) + w(F_{q+1}) + \cdots + w(F_{q+\lfloor \rho/2 \rfloor - 1}) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

where the indices are taken modulo ℓ.

Proof. Assume by way of contradiction that the above inequality does not hold for any $i \in \{0, 1, \ldots, \ell-1\}$. That is, suppose:

$$\forall i : \quad w(F_i) + w(F_{i+1}) + \cdots + w(F_{i+\lfloor \rho/2 \rfloor - 1}) > \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

1. From the definition of the sets F_i, we know that:

$$F_i \cap F_j = \emptyset \quad \text{for all } i \neq j, \quad \text{and} \quad \bigcup_{i=0}^{\ell-1} F_i = F$$

Hence,

$$\sum_{i=0}^{\ell-1} w(F_i) = w(F)$$

2. **Counting each vertex $\lfloor \rho/2 \rfloor$ times:** When we sum the weights of all such consecutive windows of length $\lfloor \rho/2 \rfloor$ across ℓ values of i, each set F_j is included in exactly $\lfloor \rho/2 \rfloor$ of these sums (i.e., it appears in that many overlapping windows). Therefore:

$$\sum_{i=0}^{\ell-1} \left(w(F_i) + \cdots + w(F_{i+\lfloor \rho/2 \rfloor - 1}) \right) = \lfloor \rho/2 \rfloor \cdot w(F)$$

3. **Contradiction:** On the other hand, if every window sum is strictly greater than $\frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$, then:

$$\sum_{i=0}^{\ell-1} \left(w(F_i) + \cdots + w(F_{i+\lfloor \rho/2 \rfloor - 1}) \right) > \ell \cdot \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F) = \lfloor \rho/2 \rfloor \cdot w(F)$$

which contradicts the equality from step 2.

4. **Conclusion:** Therefore, our assumption must be false. We conclude that:

$$\exists q \in \{0, 1, \ldots, \ell - 1\} \text{ such that } w(F_q) + \cdots + w(F_{q+\lfloor \rho/2 \rfloor - 1}) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

□

Lemma 3.10: Bound for the weight of the intersection sets (Weighted Case)

Let $F \equiv \text{OPT}$ be the optimal solution to the FVS-ρCL problem for some $\rho \in \mathbb{N}_{\geq 3}$, and let $F_{i,j} = F \cap G_{i,j}$ be the intersection of F with the subgraphs. Then there exists an integer $i^* \in \{0, 1, \ldots, \ell - 1\}$ such that:

$$\sum_j w(F_{i^*,j}) \leq w(F) + \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

Proof. When observing the construction of the subgraphs $G_{i,j}$, we note that the vertices within the overlap regions (boundary layers) are counted $\lfloor \rho/2 \rfloor$ in the total sum $\sum_j w(F_{i,j})$. The remaining (interior) vertices of each $G_{i,j}$ contribute exactly once to the total weight. This gives us:

1. Vertices on the boundary:

$$\sum_j w(F_{i,j}) = w(F) + w(\{\text{vertices on the boundary}\}) = w(F) + w(F_i) + w(F_{i+1}) + \cdots + w(F_{i+\lfloor \rho/2 \rfloor - 1})$$

for some $i \in \{0, 1, \ldots, \ell - 1\}$.

2. From Lemma 3.9, we know there exists some $i^* \in \{0, 1, \ldots, \ell - 1\}$ such that:

$$w(F_{i^*}) + w(F_{i^*+1}) + \cdots + w(F_{i^*+\lfloor \rho/2 \rfloor - 1}) \leq \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

3. Plugging this into step 1, we obtain:

$$\sum_j w(F_{i^*,j}) \leq w(F) + \frac{\lfloor \rho/2 \rfloor}{\ell} \cdot w(F)$$

□

Running time:

In Algorithm 4 which solves FVS-BCL the computation on Line 5 dominates the running time. In planar graphs steps 1 and 9 take linear time to the input. Using Courcelle's theorem [18] we argue for the running time of Line 5, which uses Corollary. 4.2 as a result. As we will say in the next section, the running time is $f(\rho, t) \cdot n^{\mathcal{O}(1)}$ for some computable function f where a tree decomposition of width t is provided. Using a result from Bodlaender [8, Theorem 83] one can incur that the treewidth of each subgraph is bounded by tw. Therefore the running time of Alg. 4 is $\mathcal{O}(f(\rho, \text{tw}) \cdot n)$ for some computable function f.

4 Establishing Fixed-Parameter Tractability for FVS-4CL and FVS-BCL

In this section we are looking for an algorithm that finds exact solutions to the FVS-4CL problem in each subgraph. For each subgraph we assume that the treewidth is bounded.

We take a logical approach that was introduced by Courcelle [18]. He shows that graph properties, that can be stated in monadic second-order logic (MSOL), can be solved in linear time for graphs of bounded treewidth. In the following two subsections we briefly explain MSOL and then apply it to our problem. We follow a survey on the matter in Satzinger [43].

4.1 MSOL - a language for graph problems

We want to introduce a language that translates graph problems to formulas, therefore reducing the problem to a satisfiability problem. Consequently, our goal is to find a formula Φ with the property:

The problem Π is solvable for a graph G *iff* $G \models \Phi$, i. e. the formula Φ is satisfied by the graph G.

Problem Π is solvable in polynomial (in this case linear) time for graphs of treewidth at most k.

Monadic second-order logic is an extension of first-order logic. Additionally to having object variables $x, y \ldots$, logical connectives $\neg, \wedge, \vee, \neg, \rightarrow, \longleftrightarrow$ and quantifiers for object variables $\forall x, \exists x$, MSOL considers sets of object variables $X, Y \ldots$, and relation to them via $\in$.

Whenever graph problems are involved, typically we express solution through vertices and edges. Additionally, as we mentioned, we can construct sets of vertices and edges. Therefore we have the following machinery:

- Object variables: *vertices*: $v_1, v_2, \ldots$ and *edges*: $e_1, e_2, \ldots$.
- Set variables: sets of vertices $V_1, V_2, \ldots$ and sets of edges $E_1, E_2, \ldots$.
- Binary relation $\in$: $\{\text{object variable}\} \times \{\text{set variable}\} \rightarrow \{0, 1\}$. Therefore $v \in V$ *iff* v is an element of the corresponding set V.
- The $Adj(e, v_i, v_j)$ relation. It detects whether edge e is an edge from vertex v_i to vertex v_j where $v_i \neq v_j$.

With this machinery, one obtains a first-order language suitable for expressing relations in a graph. To extend this to a traditional second-order logic language, one must add quantifications over set variables.

- Quantification over set variables: $\forall V_i, \forall E_i$ and $\exists V_i, \exists E_i$.

4.2 FPT Result for FVS-4CL via MSOL

Courcelle's work was extended to extended monadic second-order logic [2, 12, 19]. These extensions allow to optimize over the size and weight of a free set variable.

$$
\begin{aligned}
&\min_{F \subseteq V} |F| \\
&\quad \wedge \forall v \forall u \forall w \forall z \\
&\quad v \in (V \backslash F) \wedge u \in (V \backslash F) \wedge w \in (V \backslash F) \wedge z \in (V \backslash F) \\
&\quad \wedge v \neq w \wedge u \neq z \\
&\quad \wedge \neg((v,u) \in E \wedge (u,w) \in E \wedge (w,z) \in E \wedge (v,z) \in E)
\end{aligned} \tag{3}
$$

The minimum weight FVS-4CL can be expressed as:

$$
\begin{aligned}
&\min_{F \subseteq V} w(F) : \\
&\quad \forall v \forall u \forall w \forall z \\
&\quad v \in (V \backslash F) \wedge u \in (V \backslash F) \wedge w \in (V \backslash F) \wedge z \in (V \backslash F) \\
&\quad \wedge v \neq w \wedge u \neq z \\
&\quad \wedge \neg((v,u) \in E \wedge (u,w) \in E \wedge (w,z) \in E \wedge (v,z) \in E)
\end{aligned} \tag{4}
$$

4.3 FPT Result for Weighted FVS-BCL via MSOL

The minimum weight FVS-ρCL can be expressed as:

$$
\begin{aligned}
&\min_{F \subseteq V} w(F) : \\
&\quad \wedge \forall v_1 \forall v_2 \ldots \forall v_\rho \\
&\quad v_1 \in (V \backslash F) \wedge v_2 \in (V \backslash F) \cdots \wedge v_\rho \in (V \backslash F) \\
&\quad \wedge (v_1 \neq v_2 \wedge v_1 \neq v_3 \wedge \cdots \wedge v_1 \neq v_\rho) \\
&\quad \wedge (v_2 \neq v_1 \wedge v_2 \neq v_3 \wedge \cdots \wedge v_2 \neq v_\rho) \\
&\quad \ldots \\
&\quad \wedge (v_\rho \neq v_1 \wedge v_\rho \neq v_2 \wedge \cdots \wedge v_\rho \neq v_{\rho-1}) \\
&\quad \wedge \neg((v_1,v_2) \in E \wedge (v_2,v_3) \in E \wedge \cdots \wedge (v_{\rho-1},v_\rho) \in E \wedge (v_\rho,v_1) \in E)
\end{aligned} \tag{5}
$$

Theorem 4.1: Courcelle's theorem [18]

Assume that ϕ is a MSOL formula and G is an n-vertex graph, and there is an evaluation of all of the free variables of ϕ. Suppose, that a tree decomposition of G of width t is provided. Then there exists an algorithm that verifies that ϕ is satisfied in G in time $f(||\phi||, t) \cdot n$, for some computable function f.

Corollary 4.2: MSOL Solvability of FVS-ρCL on Bounded-Treewidth Graphs

Let $\rho \geq 3$ be a constant, and let $G = (V, E)$ be an undirected graph of treewidth at most tw, equipped with a vertex-weight function $w : V \to \mathbb{N}$. Then the minimum-weight feedback-vertex-set that breaks all ρ-cycles in G (i.e. solves FVS-ρCL) can be computed in time

$$f(\rho, \mathrm{tw}) \cdot n^{\mathcal{O}(1)},$$

where f is some computable function depending only on ρ and tw.

4.4 FPT Result for FVS-4CL via Dynamic Programming on Nice Tree Decompositions

In order to find an algorithm that solves FVS-4CL, one has to define the following notions. For a more thorough explanation of those concepts, see Bodlaender [9, Section 4.].

1. Define the notion of a *solution*. In our case for the FVS-4CL problem, the solution for a graph G is a set F, such that $G - F$ does not contain any cycles of length four.

2. Define the notion of a *partial solution*. For the subgraph $G_i = (V_i, E_i)$, partial solutions F_i are restrictions of solutions given by the subsets $F_i \subseteq V_i$.

3. Define the notion of *extension of a partial solution*. A solution F is an extension of a partial solution F_i for G_i if $F \cap V_i = F_i$.

4. Define the notion of a *characteristic* of a partial solution. For the set X_i the vertices are partitioned twofold.

   (a) The class $I \subseteq X_i$ corresponds to all vertices that are in the partial solution for the current node X_i.

   (b) The class $\mathcal{F} \subseteq X_i \times X_i$ is comprised of all of the pairs of vertices $v, u \in X_i : (v, u) \in \Phi$ if they have a common neighbour $w \in V_i \backslash X_i$, such that $w \notin F_i$.

$$ch(G_i, F_i) = (\{v \in X_i : v \in F_i\}, \{(v, u) \in X_i \times X_i : \exists w \in V_i \backslash X_i, (v, w) \in E_i, (u, w) \in E_i, w \notin F_i\})$$

The valuation of a characteristic $ch(G_i, F_i)$ is a table $c[i, I, \mathcal{F}]$. It represents the minimum weight of a partial solution F_i in G_i and $c[i, I, \mathcal{F}] \in \mathbb{N} \cup \{\infty\}$. The table keeps the value of infinity if no such set exists.

$$c[i, I, \mathcal{F}] = \min\{w(W) : W \text{ is a FVS-4CL of } G_i \wedge W \cap X_i = I\}$$

5. Define the notion of a *full set of characteristics*.

   The full set for a node X_i is a valuation for each characteristic $I \in \{0, 1\}^{|X_i|}$ and $\mathcal{F} \in \{0, 1\}^{X_i \times X_i}$. Since there are at most $\mathrm{tw}+1$ elements, there are at most $2^{(\mathrm{tw}+1)} \cdot 2^{\binom{\mathrm{tw}+1}{2}}$ entries. That yields a storage capacity of $2^{\binom{\mathrm{tw}+1}{2}+(\mathrm{tw}+1)} = 2^{\mathcal{O}(\mathrm{tw}^2)}$.

Leaf node:

For a leaf node i, we have that $X_i = \emptyset$. Hence

$$c[i, \emptyset, \emptyset] = 0$$

Introduce node:

Let i be an introduce node with child node j. We have that $X_i = X_j \cup \{v\}$ for some vertex $v \notin X_j$. There are two possible values.

$$c[i, I \cup \{v\}, \mathcal{F}] = w(v) + c[j, I, \mathcal{F}]$$

$$c[i, I, \mathcal{F}] = \begin{cases} \infty, & \text{if } \exists u, w \in X_i : (u, w) \in \mathcal{F}, u \notin I, w \notin I, (v, u) \in E_i, (v, w) \in E_i \\ c[j, I, \mathcal{F}], & \text{otherwise} \end{cases}$$

Forget node:

Let i be a forget node with child node j. We have that $X_i = X_j \backslash \{v\}$ for some vertex $v \in X_j$.

$$c[i, I, \mathcal{F}] = \min(c[j, I \cup \{v\}, \mathcal{F}], c[j, I, \mathcal{F} \cup \{(u, w) : (v, u) \in E_i, (v, w) \in E_i\}])$$

Join node:

Let i be a join node with children j_1 and j_2. Recall that $X_i = X_{j_1} = X_{j_2}$. Let $\mathcal{F}_1$ and $\mathcal{F}_2$ be the $\mathcal{F}$ set values of X_{j_1} and X_{j_2} respectively.

$$c[i, I, \mathcal{F}] = \min_{\mathcal{F}_1 \cup \mathcal{F}_2 = \mathcal{F}} \left(c[j_1, I, \mathcal{F}_1] + c[j_2, I, \mathcal{F}_2] - w(I) \right),$$

$$\text{where} \quad c[t, I, \mathcal{F}] = \infty \quad \text{if the state } (t, I, \mathcal{F}) \text{ is infeasible.}$$

A state $(t, I, \mathcal{F})$ at a *join node* t, with left child t_1 and right child t_2, is *infeasible* if there exist vertices $u, w \in X_t$ such that:

- $u, w \notin I$,
- $\{u, w\} \in \mathcal{F}_1$
- $\{u, w\} \in \mathcal{F}_2$.

In this case, since neither u nor w are in I, but both appear in the flag sets of both children, they both have forgotten adjacent vertices that have not had their weights included in the table. Therefore, a four cycle emerges where none of the vertices are included in the partial solution and thus we assign the special ∞ value to that table value.

$$c[t, I, \mathcal{F}] = \infty.$$

5 m-Stitching and Π-m-Stitching

This section introduces key concepts that facilitate the construction of certified algorithms. It begins with the definitions of *m-stitching* and Π-*m-stitching*, followed by a presentation of the main theorem from Bumpus et al. [14]. These concepts and the theorem will be used for constructing certified algorithms in the following section. Unless stated otherwise, Π is assumed to be a vertex-minimization problem. We refer to the literature in [14, 46].

5.1 FVS-4CL and 2-stitching

The restriction we imposed on the Feedback Vertex Set problem enables us to look for optimal solutions in a local space. This is what m-stitching and Π-m-stitching hint at. Here, the the variable m is a positive integer that depends on the vertex optimization problem Π. In our case this is tied to the ρ argument, i. e. the length of the cycles one is trying to break.

To introduce Π-m-stitching, it is essential to first understand m-stitching and m-stitchable problems. In the stitching operation, two solutions S_1 and S_2 are given for a vertex optimization problem, along with an induced subgraph J of G. Intuitively, the idea is to take the portion of S_1 that lies outside of J and stitch onto it a suitable part of S_2 within the subgraph J.

The formal definition is as follows:

Definition 5.1: m-stitching

Assume $m \geq 0$ is an integer, J is an induced subgraph of G, and $S_1, S_2 \subseteq V(G)$. Then we define the *m-stitch* of S_2 onto S_1 along J as the set:

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^m[J]).$$

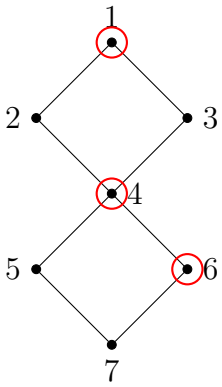

Figure 2: Vertex set S_1

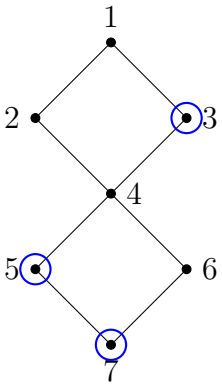

Figure 3: Vertex set S_2

In the following we show the subgraph J we have chosen and the 2-neighbourhood of the subgraph.

Also, in Fig. 5 we show the first step of the m-stitch operation, namely removing the vertices of J from the set S_1.

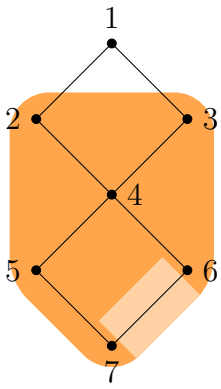

Figure 4: Subgraph J of G in light orange and $N_G^2[J]$ in darker orange

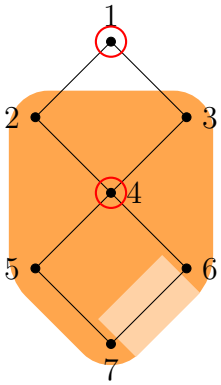

Figure 5: Remove J from S_1

What follows is the second step in obtaining set S_3 in Fig. 6 and the stitched set in Fig. 7.

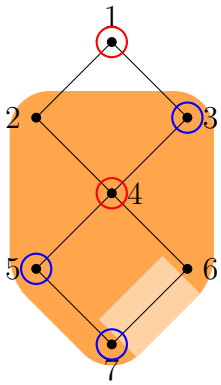

Figure 6: Add $S_2 \cap N_G^2[J]$

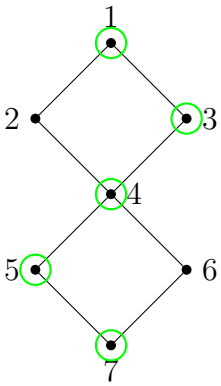

Figure 7: Set S_3

A problem Π is *m-stitchable* if the set of feasible solutions are closed under the m-stitching operator. We hope that when we take the union in 5.1 and arrive at the set S_3, this would too be a feasible solution.

Definition 5.2: m-stitchable

For an integer $m \geq 0$, induced subgraph J and feasible solutions $S_1, S_2 \subseteq V(G)$ of a vertex optimization problem Π we have that the m-stitching of S_2 onto S_1 along J, S_3 is a feasible solution to Π on G.

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^m[J]).$$

Now that we have a definition for problems Π that are m-stitchable, we introduce Π-m-stitching as defined in [14] and afterwards apply it to the current problem.

Algorithm. 5.3: Π-m-stitching

Input: an instance $(G, w : V(G) \to \mathbb{N})$ to an m-stitchable vertex optimization problem Π along with a solution S_1 and a vertex set $J \subseteq V(G)$.
Task: find a feasible solution S' to Π on G, such that for all other feasible solutions S^* it holds that $w(S') \leq w\left(S^* \oplus^2_{G,J} S_1\right)$.

Before we proceed with the main theorem that we use to construct a certified algorithm we need one more definition.

Definition 5.4: Guessable

A problem Π is *guessable* if there is an algorithm that outputs a feasible solution with no requirement for optimality in polynomial time.

The following theorem is the main result in Bumpus et al. [14, Theorem 1.7].

Theorem. 5.5: Meta-Theorem for Minor-Closed Graph Classes

Let $\mathbb{G}$ be a minor-closed graph class whose local tree-width is bounded above by a linear function of the form $g : r \to \lambda \cdot r$ (fixed $\lambda \in \mathbb{R}$ and $r \in \mathbb{N}$). If Π is a vertex-minimization problem and it holds that:

1. Π is guessable.
2. Π is m-stitchable.
3. There exists an algorithm A_Π which solves Π-m-stitching in time $f(t) \cdot |V(G)|^{\mathbb{O}(1)}$, where t = $\mathrm{tw}(G\left[N^m_G(J)\right])$ and f is a computable function;

then, for each $\epsilon > 0$ there is a $(1+\epsilon)$-certified algorithm for Π that runs in time $f(\lambda \cdot m/\epsilon) \cdot |V(G)|^{\mathbb{O}(1)}$ on any input $(G, w : V(G) \to \mathbb{N})$ with $G \in \mathbb{G}$ and polynomially-bounded weights.

The proof of theorem 5.5 is beyond the scope of this paper. In this section we only apply the three steps, mentioned to the FVS-BCL problem without generalizing to other vertex-minimization problems.

Firstly, we take a look at the guessable property.

Lemma 5.6: FVS-BCL is guessable

For any planar graph G, the set $V(G)$ is a feasible solution to FVS-BCL.

Proof. The graph $G - X$, where X is the set of all vertices $V(G)$, is the empty graph, therefore there are no 4-cycles left, after selecting that set. □

Lemma 5.7: FVS-BCL is 2-stitchable

Let (G, w) be any instance to the FVS-4CL problem, $J \subseteq V(G)$, some subset of the vertices in the graph and S_1 and S_2 any two feasible solutions to the problem in G. Then the set S_3:

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N^2_G[J]).$$

is a feasible solution to the problem.

Proof. Let the vertices $\{v_1, v_2, v_3, v_4\}$ form a 4-cycle. We inspect the possible cases for the 4-cycle $v_1v_2v_3v_4$. In order to prove 5.7 we have to prove that there exists a $v \in \{v_1, v_2, v_3, v_4\}$, such that $v \in S_3$.

- $\exists v' \in \{v_1, v_2, v_3, v_4\}$, such that $v' \in J$. We argue in three steps.
  (a) Because S_2 is a feasible solution, then $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_2$.
  (b) Because there $\exists v' \in \{v_1, v_2, v_3, v_4\} \in J$, then it also holds that $\forall v''' \in \{v_1, v_2, v_3, v_4\}, v''' \in N^2_G[J]$. From that and the previous point it follows that $v'' \in S_2 \cap N^2_G[J]$.
  (c) Finally, $S_2 \cap N^2_G[J] \subseteq S_3$, therefore $v'' \in S_3$.
- $\forall v' \in \{v_1, v_2, v_3, v_4\}$, it holds that $v' \notin J$. This case follows from the feasibility of S_1.
  (a) Because S_1 is a feasible solution, then $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_1$.
  (b) From $\forall v' \in \{v_1, v_2, v_3, v_4\} \notin J$ and the previous point, it follows that $v'' \in S_1 \backslash J$.
  (c) Because $S_1 \backslash J \subseteq S_3$, then $v'' \in S_3$.

□

Now we describe an algorithm to find a feasible solution of minimum weight.

Algorithm 5 Stitch-FVS-4CL

Require: Vertex-weighted planar graph $(G, w : V(G) \to \mathbb{N})$, a feasible solution S_1 on G and a vertex set $J \subseteq V(G)$.

Ensure: a feasible solution S' on G, such that for all other feasible solutions S^*, we have $w(S') \leq w(S^* \oplus^2_{G,J} S_1)$.

1: Let $H \leftarrow G[N^2_G[J] \backslash (S_1 \backslash J)]$.
2: Let S_2 be the output of algorithm A on input (H, w).
3: **if** $w(S_2 \oplus^2_{G,J} S_1) < w(S_1)$ **then**
4: **return** $S_2 \oplus^2_{G,J} S_1$
5: **else**
6: **return** S_1
7: **end if**

The algorithm A on Line 5 refers to using Corollary 4.2 as a result.

Lemma 5.8: FVS-4CL-2-stitching

Let A be an algorithm that solves FVS-4CL on any vertex-weighted instance $(G, w : V(G) \to \mathbb{N})$ in time $f(t) \cdot |V(G)|^{\mathcal{O}(1)}$ where t is the treewidth of G. Then Algorithm 5 solves FVS-4CL-2-stitching in time $f(\text{tw}) \cdot n^{\mathcal{O}(1)}$ where tw is the treewidth of $N^2_G[J]$. In the end of the lemma we argue about the exact running time of the algorithm.

Note that it is not obvious that the set $S_2 \oplus^2_{G,J} S_1$ is a feasible solution in G as S_2 is not necessarily a feasible solution in G. If the last was the case, then the feasibility of the $S_2 \oplus^2_{G,J} S_1$ would follow from Lemma 5.7. Therefore we first prove feasibility of $S_2 \oplus^2_{G,J} S_1$. Afterwards, we prove that it is of minimum weight.

Proof. We prove the feasibility of $S_2 \oplus^2_{G,J} S_1$ by case analysis on any four vertices that form a 4-cycle. Remember that $S_2 \oplus^2_{G,J} S_1 = (S_1 \backslash J) \cup (S_2 \cap N^2_G[J])$.

1. $\forall v' \in \{v_1, v_2, v_3, v_4\} : v' \in V(G) \backslash J$.

   Firstly, because of the feasibility of S_1, there $\exists v'' \in \{v_1, v_2, v_3, v_4\} : v'' \in S_1$.

   Secondly, $\forall v' \in \{v_1, v_2, v_3, v_4\}$, it holds that $v' \in V(G) \backslash J$.

   Finally from the first two conditions, using the property $A \backslash B = A \cap B^c$, it holds that $v'' \in S_1 \backslash J$. From this and $S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$, it follows that $S_2 \oplus^2_{G,J} S_1$ is feasible in G.

   In Fig. 8 we show (a part of) an example graph G and the feasible solution S_1 for it. In Fig. 9, one can see that 4-cycles that do not have vertices in J, are a part of $S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$.

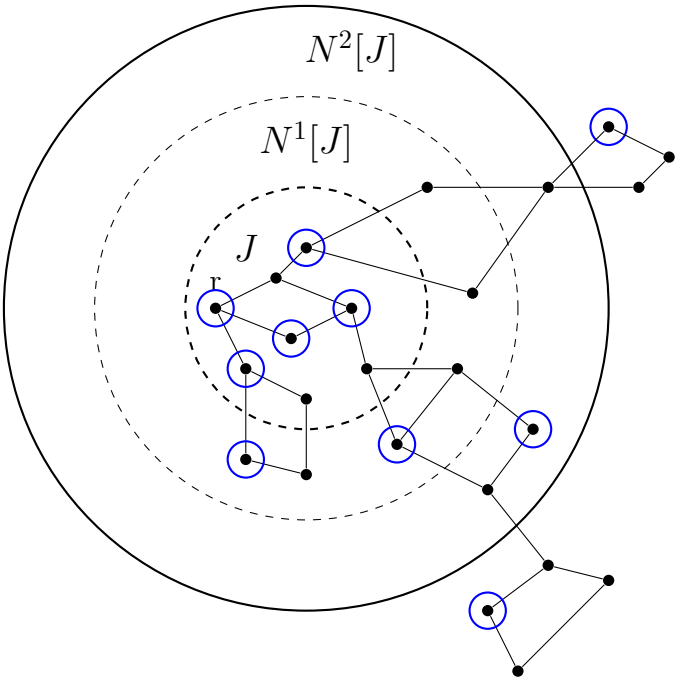

Figure 8: Sets $J \subseteq N^1_G[J] \subseteq N^2_G[J] \subseteq G$; Feasible solution S_1 colored in blue.

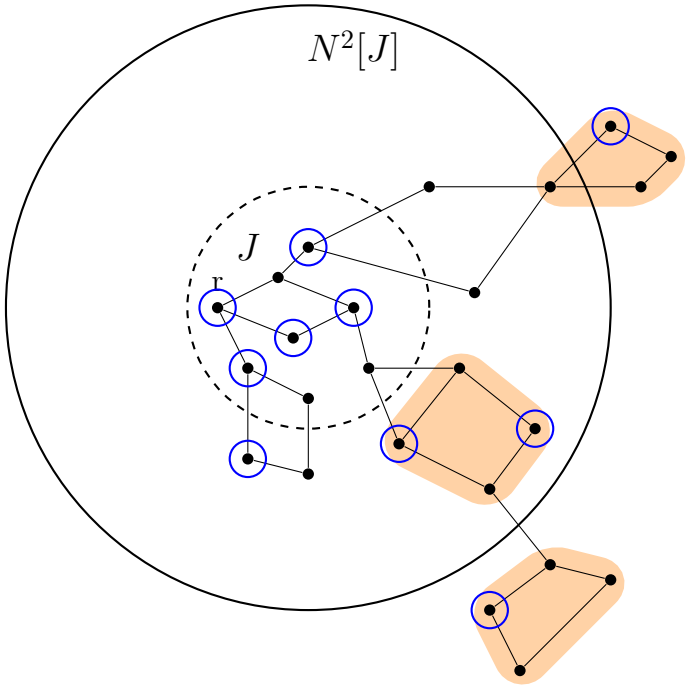

Figure 9: $\forall v \in \{v_1, v_2, v_3, v_4\} : v \notin J$

2. $\exists v' \in \{v_1, v_2, v_3, v_4\} : v' \in J$. This also means that in this case $\forall v' \in \{v_1, v_2, v_3, v_4\} : v' \in N^2_G[J]$. There are two cases to consider again. We know there $\exists v'' \in \{v_1, v_2, v_3, v_4\}$, such that $v'' \in S_1$, we still have not considered whether or not it is a part of J.

   (a) $\exists v \in S_1 : v \notin J$:

   In this case we can directly apply the previous point. From the properties of the set difference operator we get: $v \in S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$. Consult Fig. 10 to see that there exists a vertex $v_{1,4}$ that is a part of the 4-cycle, $v_{1,4} \notin J$, therefore $v_{1,4} \in S_1 \backslash J \subseteq S_2 \oplus^2_{G,J} S_1$. This means that in the new set $v_{1,4}$ will break that cycle.

   (b) $\forall v \in S_1 : v \in J$:

   In Line 1 of Algorithm 5, we have the following construction of $H \leftarrow G[N^2_G[J] \backslash (S_1 \backslash J)]$.

$$V(H) = N_G^2[J] \backslash (S_1 \backslash J) \tag{6}$$
$$= N_G^2[J] \cap J \cup (N_G^2[J] \backslash S_1) \tag{7}$$
$$= J \cup (N_G^2[J] \backslash S_1) \tag{8}$$

However, $\forall v \in \{v_1, v_2, v_3, v_4\} : v \in S_1 \wedge v \in J$. Therefore $\{v_1, v_2, v_3, v_4\} \subseteq V(H)$. This means that in Line 2 of Algorithm 5 we will add some $v^* \in \{v_1, v_2, v_3, v_4\}$ to S_2.

Therefore $v \in S_2$, which has the same meaning as $v \in S_2 \cap N_G^2[J]$, and because $S_2 \cap N_G^2[J] \subseteq S_2 \oplus_{G,J}^2 S_1$, we get $v \in S_2 \oplus_{G,J}^2 S_1$.

Consult Fig. 11.

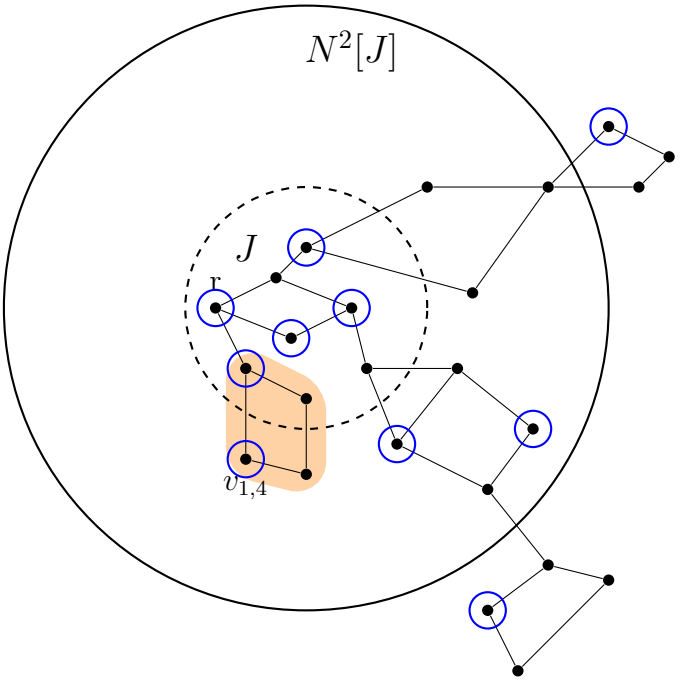

Figure 10: $\exists v \in S_1 : v \notin J$ (Remember we are in the case where $\exists v \in J$)

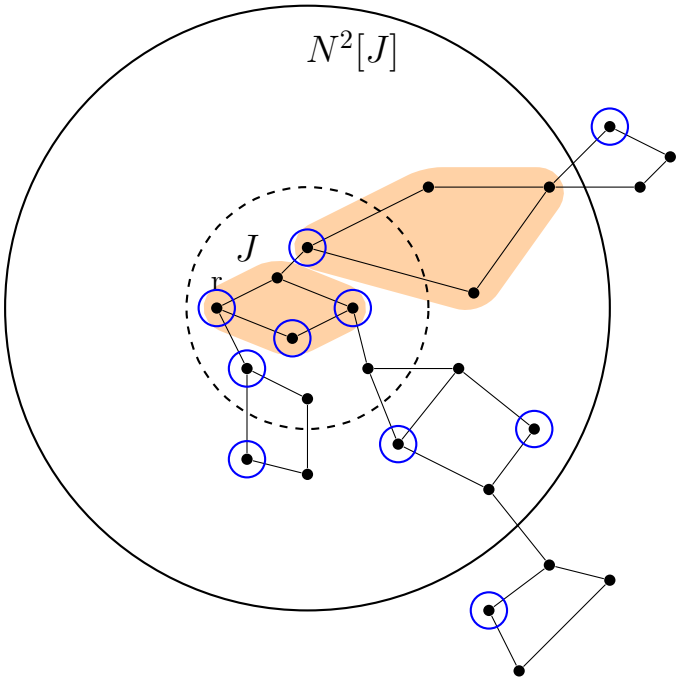

Figure 11: $\forall v \in S_1 : v \in J$

The proof of optimality of the set $S_2 \oplus_{G,J}^2 S_1$ is similar to the one in Bumpus et al.[14, Lemma 3.2]. It is important to note that we can use the same proof, because of a property of the FVS-4CL problem, namely that a 4-cycle breaking set is closed under taking induced subgraphs.

Remember, we are still trying to prove what we mentioned in Algorithm 5.3, namely that some feasible solution S' (which in our case is the $S_2 \oplus_{G,J}^2 S_1$), is of minimum weight, that is, for all other feasible solutions S^*, it holds that $w(S') \leq w(S^* \oplus_{G,J}^2 S_1)$.

Assume by way of contradiction that there is a feasible solution S_3 for G, such that $w(S_3 \oplus_{G,J}^2 S_1) < w(S_2 \oplus_{G,J}^2 S_1)$. Then it would follow that:

$$
\begin{aligned}
w|_{V(F)}(S_2) &= w(S_2) && \text{since } S_2 \subseteq V(F) \\
&= w(S_2 \cap N_G^2[J]) && \text{since } V(F) \subseteq N_G^2[J] \\
&= w(S_1 \setminus J) + w(S_2 \cap N_G^2[J]) - w(S_1 \setminus J) \\
&= w((S_1 \setminus J) \cup (S_2 \cap N_G^2[J])) - w(S_1 \setminus J) && \text{since } V(F) \cap (S_1 \setminus J) = \emptyset \\
&= w(S_2 \oplus_{G,J}^2 S_1) - w(S_1 \setminus J) && \text{by definition of stitch} \\
&> w(S_3 \oplus_{G,J}^2 S_1) - w(S_1 \setminus J) && \text{by assumption on } S_3 \\
&= w((S_1 \setminus J) \cup (S_3 \cap N_G^2[J])) - w(S_1 \setminus J) && \text{by definition of stitch} \\
&= w((S_3 \cap N_G^2[J]) \setminus (S_1 \setminus J)) && \text{since } w(A \cup B) - w(A) = w(B \setminus A) \\
&= w(S_3 \cap (N_G^2[J] \setminus (S_1 \setminus J))) && \text{since } (A \cap B) \setminus C = A \cap (B \setminus C) \\
&= w(S_3 \cap V(F)) && \text{by definition of } F \\
&= w|_{V(F)}(S_3 \cap V(F))
\end{aligned}
$$

□

Running time:

The running time of the algorithm is dominated by the call of algorithm A. For tw, the treewidth of $N_G^2[J]$, the running time is $2^{\mathcal{O}(\text{tw}^2)} \cdot n^{\mathcal{O}(1)}$. Here we use the dynamic programming algorithm (4.4).

5.2 FVS-BCL and $(\rho/2)$-stitching

In this subsection, we extend the results for FVS-4CL to the more general problem. In earlier sections, we showed that FVS-BCL admits an EPTAS by observing that $\lfloor \frac{\rho}{2} \rfloor$ layers of overlap between two subgraphs are sufficient for a ρ-cycle to be entirely contained within one of the subgraphs, possibly including the overlapping region (see Fig. 1, 8). When applying the m-stitching technique, the reader should understand that the two relevant subgraphs are $G[V(G) \setminus J]$ and $G[J]$. If we construct an algorithm similar to Algorithm 5 but set the parameter m to $\lfloor \frac{\rho}{2} \rfloor$ instead of 2, the resulting algorithm solves FVS-BCL-stitching in time $f(\text{tw}) \cdot n^{\mathcal{O}(1)}$ where tw is the treewidth of $N_G^{\lfloor \frac{\rho}{2} \rfloor}[J]$. It is important to note that a ρ-cycle breaking set is closed under taking induced subgraphs. This is another reason we can construct such an algorithm and do not have to take a different approach as we will do with the r-Dominating Set.

From the arguments in the previous paragraph we can infer that:

1. FVS-BCL is $\lfloor \frac{\rho}{2} \rfloor$-stitchable.
2. There exists an algorithm $A_{\text{FVS-BCL}}$ which solves FVS-BCL-$\lfloor \frac{\rho}{2} \rfloor$-stitching in time $f(\text{tw}) \cdot n^{\mathcal{O}(1)}$ where tw is the treewidth of any $N_G^{\lfloor \frac{\rho}{2} \rfloor}[J]$ and f is some computable function.

The problem is also trivially guessable. Like in the FVS-4CL, taking the set $V(G)$, would break all ρ-cycles in a graph.

Running time:

For some computable function f, constant ρ and tw, treewidth of $N_G^{\lfloor \frac{\rho}{2} \rfloor}[J]$, the algorithm we described solves FVS-BCL-ρ-stitching in time $f(\text{tw}) \cdot n^{\mathcal{O}(1)}$.

5.3 r-Dominating Set and $(r+1)$-stitching

In this subsection we will use the definitions and terminology we have established in the previous section and look at another problem, namely the r-Dominating Set problem. We will argue that in the case of that problem the parameter m has to be set to $r+1$ in order to maintain feasibility when constructing new sets. Afterwards we will prove that when constructing new sets, one can find a feasible solution which is also optimal. We follow the steps we established in the previous section.

Firstly, prove that the problem is guessable and m-stitchable. Secondly, devise an algorithm $A_{\text{r-DS}}$ that solves the problem in FPT. Finally, using Theorem 5.5, prove that the problem is certified.

Lemma 5.9: r-DS is guessable

For any planar graph G, the set $V(G)$ is a feasible solution to r-DS.

Proof. Let X be the set of all vertices in G, $X \leftarrow V(G)$. Therefore, it is trivially true that $N_G^r[X] = V(G)$. □

Lemma 5.10: r-DS is $(r+1)$-stitchable

Let (G, w) be any instance to the r-DS problem, $J \subseteq V(G)$, some subset of the vertices in the graph and S_1 and S_2 any two feasible solutions to the problem in G. Then the set S_3:

$$S_3 := (S_1 \backslash J) \cup (S_2 \cap N_G^{r+1}[J]).$$

is a feasible solution to the problem.

Proof. For a vertex $v \in V(G)$, we have the following cases:

1. $v \in V(G) \backslash N_G^{r+1}[J]$. Because S_1 is a feasible solution, it is clear that v will be dominated by some vertex u in $S_1 \backslash J$. Since $S_1 \backslash J \subseteq S_3$, then $u \in S_3$.
2. $v \in N_G^{r+1}[J]$. Because S_2 is a feasible solution, it is clear that $S_2 \cap N_G^{r+1}[J]$ contains a vertex u which dominates v. Since $S_2 \cap N_G^{r+1}[J] \subseteq S_3$, then $u \in S_3$.

In both cases we have argued that there $\exists u \in S_3$, such that for any vertex $v \in V(G)$, u dominates v.

□

Now we describe an algorithm to find a feasible solution of minimum weight.

Algorithm 6 Stitch-r-DS

Require: Vertex-weighted planar graph $(G, w : V(G) \to \mathbb{N})$, a feasible solution of r-DS S_1 on G and a vertex set $J \subseteq V(G)$.

Ensure: a feasible solution S' on G, such that for all other feasible solutions S^*, we have $w(S') \leq w(S^* \oplus^{r+1}_{G,J} S_1)$.

1: Let $H \leftarrow G[N^{r+1}_G[J]] \cup \{q\}$, where for vertex q it holds that $N_H(q) := N_G(N^r_G[J])$ (Vertex q is adjacent to the vertices at distance exactly $r+1$ from J).

2:
$$w_H(x) = \begin{cases} 0 & \text{if } x \in (S_1 \setminus J) \cup \{q\} \\ w(x) & \text{otherwise} \end{cases}$$

3: Let S'_2 be the output of algorithm A on input (H, w).

4: Let $S_2 = S'_2 \setminus \{q\}$.

5: **if** $w(S_2 \oplus^{r+1}_{G,J} S_1) < w(S_1)$ **then**

6: **return** $S_2 \oplus^{r+1}_{G,J} S_1$

7: **else**

8: **return** S_1

9: **end if**

Lemma 5.11: r-DS-$(r+1)$-stitching

Let A be an algorithm that solves r-stitching on any vertex-weighted instance $(G, w : V(G) \to \mathbb{N})$ in time $f(t) \cdot |V(G)|^{\mathcal{O}(1)}$ where t is the treewidth of G. Then Algorithm 6 solves r-DS in time $f(\text{tw}) \cdot V(G)^{\mathcal{O}(1)}$ where tw is the treewidth of $(N^{r+1}_G[J])$.

The proof of Lemma 5.11 is outside of the scope of this paper as it is similar to the proof of Lemma 5.8. The reader can find a similar proof in Bumpus et al. [14, Lemma 3.3].

Running time:

For some constant r and tw, the treewidth of $N^{r+1}_G[J]$, the running time of the algorithm is: $(2r+1)^{\text{tw}+1} \cdot \text{tw}^2 \cdot n^{\mathcal{O}(1)}$.

6 Certified Algorithms

In this section, we initiate the study of certified algorithms for FVS-4CL and the more general FVS-BCL problem. We conclude with a discussion on extending these techniques to the r-Dominating Set problem.

In the previous section, we established a framework and proved several properties that apply to all three problems. This allows us to now claim the existence of $(1+\epsilon)$-certified algorithms for each of them. For FVS-4CL, we illustrate how to derive such an algorithm by adapting our earlier techniques, though we do not explicitly prove that the solutions are $(1+\epsilon)$-certified. For the remaining problems, we invoke Theorem 5.5 directly to obtain the desired guarantees.

Instead of directly invoking Theorem 5.5 to assert the existence of a $(1+\epsilon)$-certified algorithm for FVS-4CL, we explicitly present it. While a formal proof that the algorithm maps instances to $(1+\epsilon)$-certified solutions lies outside the scope of this paper, our aim is to provide a clear and instructive presentation. This should help illustrate how such certified algorithms can be derived. The algorithm is adapted from Bumpus et al. [14, Section 4.2.1].

There are three main steps in Algorithm 7. In Line 1 we only define variables that help with the computation.

Algorithm 7 $(1+\epsilon)$-certified algorithm for FVS-4CL

Require: Vertex-weighted planar graph $(G, w : V(G) \to \mathbb{N})$, $\epsilon > 0$.

Ensure: a vertex set $S^* \subseteq V(G)$ and a $(1+\epsilon)$-perturbation w' of w, such that S^* is an optimal solution (in this case for FVS-4CL on (G, w'))

1: $\kappa \leftarrow \lceil \frac{2m}{\epsilon} \rceil + 2m$ ($m \leftarrow 2$ in this case); Let S^* be a feasible solution (we proved that FVS-4CL is guessable)
2: Perform BFS from an arbitrary vertex r
3: **while** there exists a subgraph $J_{\kappa-2m}$ of width $\kappa - 2m$ such that $w_A((G,w), S^*, J_{\kappa-2m}) < w(S^*)$ **do**
4: Replace S^* with $A((G,w), S^*, J_{\kappa-2m})$
5: **end while**
6: **return** (S^*, w'), where $w' : V(G) \to \mathbb{R}^+$ is defined by:

$$w'(x) = \begin{cases} w(x) & \text{if } x \in S^* \\ (1+\epsilon)w(x) & \text{otherwise} \end{cases}$$

The first stage simply runs a BFS from a root node. This layer decomposition is used in further steps to facilitate the correct inducing of subgraphs.

In the second stage, we compute a feasible solution S^*, set $J_{\kappa-2m}$[1] and improve that solution whilst a better one can be found. This step incurs additional running time. Note that, for $W = max_{v \in V(G)} w(v)$ the number of times we can find a strictly better solution than the current one is bounded by $W \cdot n$. This

[1] A word on the argument κ and the value that we set it to. In our case, for the FVS-4CL problem, m is equal to two. Also, $\kappa = \lceil \frac{2m}{\epsilon} \rceil + 2m \geq 5$. Notice that the set $J_{\kappa-2m}$ is of width $\kappa - 2m$, therefore the m-neighbourhood (2-neighbourhood) is of width $\kappa - 2m + 2m = \kappa$. Therefore, in our case, the graphs within we stitch are of width $\kappa = \lceil \frac{2m}{\epsilon} \rceil + 2m = \lceil \frac{4}{\epsilon} \rceil + 4 \geq 5$. Take for example vertex $v_{2,1}$ in Fig. 12. In order to find every cycle that it is a part of, one would need to consider the layer that it is in, two layers to the left and two layers to the right.

is because our weights are integers and the value of a solution cannot be less than 0. We now understand why the weights have to be polynomially-bounded as is stated in the main theorem.

In the third and final step we obtain a $(1+\epsilon)$-perturbation of w. The weight function w' is derived by maximally increasing every vertex that is not in the solution S^*. Intuitively, this is the reason that our solution S^* is an optimal instance of FVS-4CL in (G, w').

Notice that in Algorithm 7 there is not anything FVS-4CL specific, apart from the three properties, mentioned in the main theorem. Therefore, this algorithm can be used for any problem that satisfies those structural properties.

Finally we invoke Theorem 5.5 to get:

Corollary 6.1: $(1+\epsilon)$-Certified Algorithm for Planar Weighted FVS and r-Domination

For a planar graph G and a weighted function $w : V(G) \to \mathbb{N}$ (polynomially-bounded weights) and $\epsilon > 0$ there are $(1+\epsilon)$-certified algorithms solving FVS-4CL, FVS-BCL and r-Dominating Set.

The algorithms admit the following worst-case running times, where tw is the treewidth of the graph G.

1. FVS-4CL: $2^{\cdot\mathcal{O}((\frac{2}{\epsilon})^2)} \cdot n^{\mathcal{O}(1)}$.
2. FVS-BCL: $f(\lfloor \frac{\rho}{2\cdot\epsilon} \rfloor) \cdot n^{\mathcal{O}(1)}$ for some computable function f.
3. r-Dominating Set: $(2r+1)^{\mathcal{O}(\frac{r}{\epsilon})} \cdot \left(\frac{r}{\epsilon}\right)^2 \cdot n$ for a constant r.

This concludes the thesis.

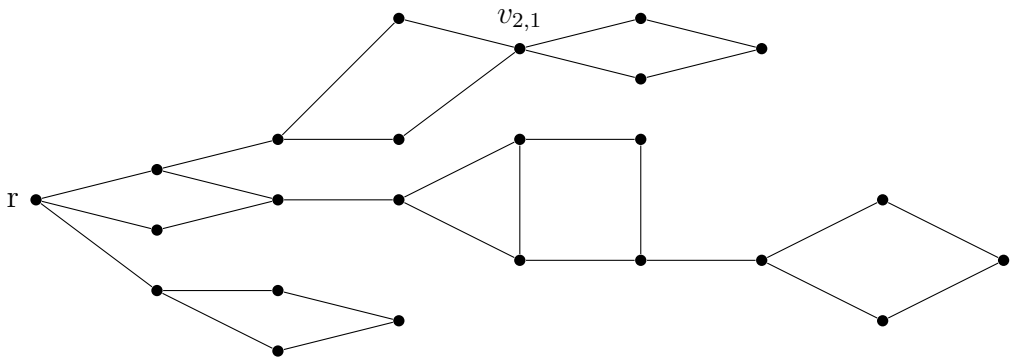

Figure 12: Layer decomposition of graph in Fig. 8

7 Conclusion

The primary goal of this thesis was to explore the weighted version of the Feedback Vertex Set with Bounded Cycle Length (FVS-BCL) problem in planar graphs and develop approximate solutions. Unlike the classical Feedback Vertex Set problem, the FVS-BCL variant has received comparatively little attention, despite its potential relevance in applications. To tackle this problem, we employed a combination of established methods, such as Baker's outerplanarity technique, and more recent innovations, including the construction of certified algorithms for instances of polynomially-bounded weights. This approach was chosen not only to contribute new insights into the FVS-BCL problem but also to provide a broader understanding of how hard problems can be addressed by leveraging local structural properties.

The key findings of this thesis can be grouped into three main contributions. First, we demonstrated that the FVS-BCL problem is fixed-parameter tractable when parameterized by treewidth in Section 4. This result was crucial for enabling the subsequent developments. Second, by applying Baker's technique, we established the existence of an efficient polynomial-time approximation scheme (EPTAS) for the FVS-BCL problem in planar graphs in Section 3. Lastly, we presented an algorithm for constructing $(1+\epsilon)$-certified solutions, providing not only approximate answers but also formal guarantees on their quality in Section 6.

The results of this thesis can be interpreted as an extension of existing work, providing new insights and positive results for the FVS-BCL problem. Beyond the theoretical results, this research has practical implications for areas where constraints on cycle length are critical, such as in network reliability and circuit design. Moreover, the techniques developed here, particularly the use of certified algorithms, may serve as a foundation for future work on similarly structured graph problems.

One limitation of this work is that, although we aimed to generalize the positive results obtained for the FVS-BCL problem to the more general Feedback Vertex Set problem on weighted planar graphs, we were unable to establish a direct connection. While we successfully developed an efficient PTAS for the FVS-BCL variant, the existence of such an approximation scheme for the general weighted FVS problem in planar graphs remains an open question. In particular, it would be valuable to investigate how many cycles a ρ-cycle breaking set intersects.

There are a couple of directions for future research. The first one relates to the tractability results in Subsection 4.4. Improving the $2^{\mathcal{O}(\mathrm{tw}^2)} \cdot n$ running time could be explored by a deploying some existing techniques. The traditional dynamic programming techniques for FVS yield $2^{\mathcal{O}(\mathrm{tw} \cdot \log \mathrm{tw})} \cdot n^{\mathcal{O}(1)}$. Furthermore, Cygan et al. [21] developed the Cut & Count technique which obtained a $3^{\mathcal{O}(\mathrm{tw})} \cdot n^{\mathcal{O}(1)}$ randomized algorithm and Bodlaender et al. [10] showed a deterministic $2^{\mathcal{O}(\mathrm{tw})} \cdot n^{\mathcal{O}(1)}$ rank-based approach. Any of these should serve as motivation to improve the algorithm in this thesis. Another promising direction for future research is improving the certified algorithm for FVS-BCL. In particular, it would be valuable to develop an approach that eliminates the current reliance on the polynomially-bounded weights constraint. At present, this assumption is essential for achieving $(1+\epsilon)$-certified solutions.

References

[1] Haris Angelidakis et al. “Bilu-linial stability, certified algorithms and the independent set problem”. In: *arXiv preprint arXiv:1810.08414* (2018) (cit. on pp. 3, 4).

[2] Stefan Arnborg, Jens Lagergren, and Detlef Seese. “Easy problems for tree-decomposable graphs”. In: *Journal of Algorithms* 12.2 (1991), pp. 308–340 (cit. on p. 25).

[3] Vineet Bafna, Piotr Berman, and Toshihiro Fujito. “Constant ratio approximations of the weighted feedback vertex set problem for undirected graphs”. In: *Algorithms and Computations: 6th International Symposium, ISAAC'95, 1995 Proceedings 6*. Springer. 1995, pp. 142–151 (cit. on p. 3).

[4] Brenda S. Baker. “Approximation algorithms for NP-complete problems on planar graphs”. In: *J. ACM* 41.1 (Jan. 1994), pp. 153–180. DOI: 10.1145/174644.174650 (cit. on pp. 3, 5, 8).

[5] Reuven Bar-Yehuda et al. “Approximation algorithms for the feedback vertex set problem with applications to constraint satisfaction and Bayesian inference”. In: *SIAM Journal on Computing* 27.4 (1998), pp. 942–959 (cit. on p. 3).

[6] Ann Becker and Dan Geiger. “Optimization of Pearl's method of conditioning and greedy-like approximation algorithms for the vertex feedback set problem”. In: *Artificial Intelligence* 83.1 (1996), pp. 167–188 (cit. on p. 3).

[7] Yonatan Bilu and Nathan Linial. “Are stable instances easy?” In: *Combinatorics, Probability and Computing* 21.5 (2012), pp. 643–660 (cit. on pp. 3, 10).

[8] Hans L Bodlaender. “A partial k-arboretum of graphs with bounded treewidth”. In: *Theoretical Computer Science* 209.1-2 (1998), pp. 1–45 (cit. on pp. 9, 17, 24).

[9] Hans L Bodlaender. “Treewidth: Algorithmic techniques and results”. In: *International symposium on mathematical foundations of computer science*. Springer. 1997, pp. 19–36 (cit. on p. 27).

[10] Hans L Bodlaender et al. “Deterministic single exponential time algorithms for connectivity problems parameterized by treewidth”. In: *Information and Computation* 243 (2015), pp. 86–111 (cit. on p. 40).

[11] John Adrian Bondy, Uppaluri Siva Ramachandra Murty, et al. *Graph Theory with Applications*. Vol. 290. Macmillan London, 1976 (cit. on p. 5).

[12] Richard B Borie, R Gary Parker, and Craig A Tovey. “Deterministic dcomposition of recursive graph classes”. In: *SIAM Journal on Discrete Mathematics* 4.4 (1991), pp. 481–501 (cit. on p. 25).

[13] Glencora Borradaile and Hung Le. “Optimal dynamic program for r-domination problems over tree decompositions”. In: *arXiv preprint arXiv:1502.00716* (2015) (cit. on p. 11).

[14] Benjamin Merlin Bumpus, Bart M. P. Jansen, and Jaime Venne. “Fixed-Parameter Tractable Certified Algorithms for Covering and Dominating in Planar Graphs and Beyond”. In: *19th Scandinavian Symposium and Workshops on Algorithm Theory (SWAT 2024)*. Ed. by Hans L. Bodlaender. Vol. 294. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 19:1–19:16. ISBN: 978-3-95977-318-8. DOI: 10.4230/LIPIcs.SWAT.2024.19 (cit. on pp. 3, 4, 9, 29–31, 34, 37, 38).

[15] Marco Cesati and Luca Trevisan. “On the efficiency of polynomial time approximation schemes”. In: *Information Processing Letters* 64.4 (1997), pp. 165–171 (cit. on p. 7).

[16] Fabián A Chudak et al. "A primal–dual interpretation of two 2-approximation algorithms for the feedback vertex set problem in undirected graphs". In: *Operations Research Letters* 22.4-5 (1998), pp. 111–118 (cit. on p. 3).

[17] Vincent Cohen-Addad et al. "Approximating connectivity domination in weighted bounded-genus graphs". In: *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*. 2016, pp. 584–597 (cit. on p. 3).

[18] Bruno Courcelle. "The monadic second-order logic of graphs. I. Recognizable sets of finite graphs". In: *Information and Computation* 85.1 (1990), pp. 12–75 (cit. on pp. 24–26).

[19] Bruno Courcelle and Mohamed Mosbah. "Monadic second-order evaluations on tree-decomposable graphs". In: *Theoretical Computer Science* 109.1-2 (1993), pp. 49–82 (cit. on p. 25).

[20] Marek Cygan et al. *Parameterized Algorithms*. Vol. 5. 4. Springer, 2015 (cit. on p. 5).

[21] Marek Cygan et al. "Solving connectivity problems parameterized by treewidth in single exponential time". In: *ACM Transactions on Algorithms (TALG)* 18.2 (2022), pp. 1–31 (cit. on p. 40).

[22] Erik Demaine et al. *Lecture 9: Approximation schemes in minor-free graphs I*. Lecture notes. 2011. URL: `https://courses.csail.mit.edu/6.889/fall11/lectures/L09.html` (cit. on p. 11).

[23] Erik D Demaine and Mohammad Taghi Hajiaghayi. "Bidimensionality: new connections between FPT algorithms and PTASs." In: *Symposium on Discrete Algorithms*. Vol. 5. 2005, pp. 590–601 (cit. on p. 3).

[24] Erik D Demaine and Mohammad Taghi Hajiaghayi. "Equivalence of local treewidth and linear local treewidth and its algorithmic applications". In: (2003) (cit. on p. 9).

[25] Reinhard Diestel. *Graph Theory*. 6th ed. Vol. 173. Graduate Texts in Mathematics. Springer-Verlag, 2025. ISBN: 978-3-662-53621-6 (cit. on p. 9).

[26] David Eppstein. "Diameter and treewidth in minor-closed graph families". In: *Algorithmica* 27 (2000), pp. 275–291 (cit. on p. 9).

[27] Paola Festa, Panos M Pardalos, and Mauricio G C. Resende. "Feedback set problems". In: *Encyclopedia of optimization*. Springer, 2012, pp. 1–13 (cit. on p. 3).

[28] Michel X Goemans and David P Williamson. "Primal-dual approximation algorithms for feedback problems in planar graphs". In: *Combinatorica* (1998) (cit. on p. 3).

[29] Rajesh Gupta, Rajiv Gupta, and Melvin A Breuer. "BALLAST: A methodology for partial scan design". In: *1989 The Nineteenth International Symposium on Fault-Tolerant Computing. Digest of Papers*. IEEE Computer Society. 1989, pp. 118–125 (cit. on p. 3).

[30] Rudolf Halin. "S-functions for graphs". In: *Journal of Geometry* 8 (1976), pp. 171–186 (cit. on p. 9).

[31] Donald B Johnson. "Finding all the elementary circuits of a directed graph". In: *SIAM Journal on Computing* 4.1 (1975), pp. 77–84 (cit. on p. 3).

[32] Richard M. Karp. "Reducibility among combinatorial problems". In: ed. by Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger. Springer US, 1972, pp. 85–103. DOI: `10.1007/978-1-4684-2001-2_9` (cit. on p. 3).

[33] Jon Kleinberg and Amit Kumar. "Wavelength conversion in optical networks". In: *Journal of algorithms* 38.1 (2001), pp. 25–50 (cit. on p. 3).

[34] Bernhard Korte and Jens Vygen. "Combinatorial Optimization: Theory and Algorithms". In: 2007. URL: https://api.semanticscholar.org/CorpusID:265900003 (cit. on pp. 5, 7, 10).

[35] Arno Kunzmann and Hans-Joachim Wunderlich. "An analytical approach to the partial scan problem". In: *Journal of Electronic Testing* 1 (1990), pp. 163–174 (cit. on p. 3).

[36] Hung Le and Baigong Zheng. "Local search is a PTAS for feedback vertex set in minor-free graphs". In: *Theoretical Computer Science* 838 (2020), pp. 17–24 (cit. on p. 3).

[37] Errol L Lloyd, Mary Lou Soffa, and Ching-Chy Wang. "On locating minimum feedback vertex sets". In: *Journal of Computer and System Sciences* 37.3 (1988), pp. 292–311 (cit. on p. 3).

[38] Flaminia L Luccio et al. "Almost exact minimum feedback vertex set in meshes and butterflies". In: *Inf. Process. Lett.* 66.2 (1998), pp. 59–64 (cit. on p. 3).

[39] Konstantin Makarychev. "13 Bilu-Linial Stability". In: *Perturbations, Optimization, and Statistics* (2017), p. 375 (cit. on p. 10).

[40] Konstantin Makarychev and Yury Makarychev. "Perturbation resilience". In: *Beyond the Worst-Case Analysis of Algorithms*. Ed. by TimEditor Roughgarden. Cambridge University Press, 2021, pp. 95–119 (cit. on pp. 3, 4, 10).

[41] Marjan Marzban and Qian-Ping Gu. "Computational study on a PTAS for Planar Dominating Set Problem". In: *Algorithms* 6.1 (2013), pp. 43–59. ISSN: 1999-4893. DOI: 10.3390/a6010043 (cit. on p. 8).

[42] Neil Robertson and P.D Seymour. "Graph minors. II. Algorithmic aspects of tree-width". In: *Journal of Algorithms* 7.3 (1986), pp. 309–322. DOI: https://doi.org/10.1016/0196-6774(86)90023-4 (cit. on p. 9).

[43] Christina Satzinger. "On Tree-Decompositions and their Algorithmic Implications for Bounded-Treewidth Graphs". MA thesis. TU Wien, Österreich, 2014 (cit. on p. 25).

[44] Alan C. Shaw. "Software descriptions with flow expressions". In: *IEEE Transactions on Software Engineering* 3 (1978), pp. 242–254 (cit. on p. 3).

[45] Johan MM Van Rooij, Hans L Bodlaender, and Peter Rossmanith. "Dynamic programming on tree decompositions using generalised fast subset convolution". In: *Algorithms-ESA 2009: 17th Annual European Symposium, Copenhagen, Denmark, September 7-9, 2009. Proceedings 17*. Springer. 2009, pp. 566–577 (cit. on p. 11).

[46] JC Venne. "Designing (1)-certified algorithms for planar optimiza-tion problems with local feasibility properties". In: (2022) (cit. on p. 29).

[47] David P Williamson and David B Shmoys. *The design of Approximation Algorithms*. Cambridge university press, 2011 (cit. on p. 5).

A Appendix

A.1 Example one graph and Nice Tree Decomposition

Here we provide a detailed simulation of the dynamic programming algorithm.

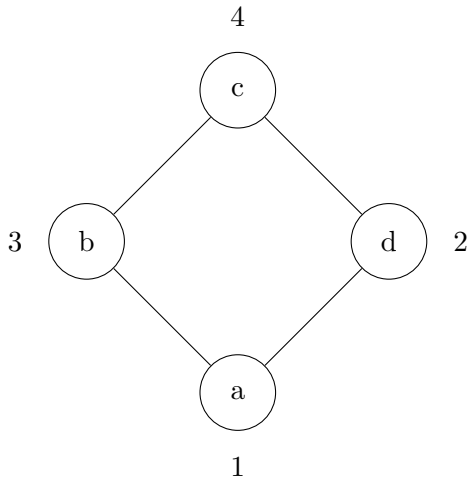

Figure 13: Weighted Rhombus Graph G

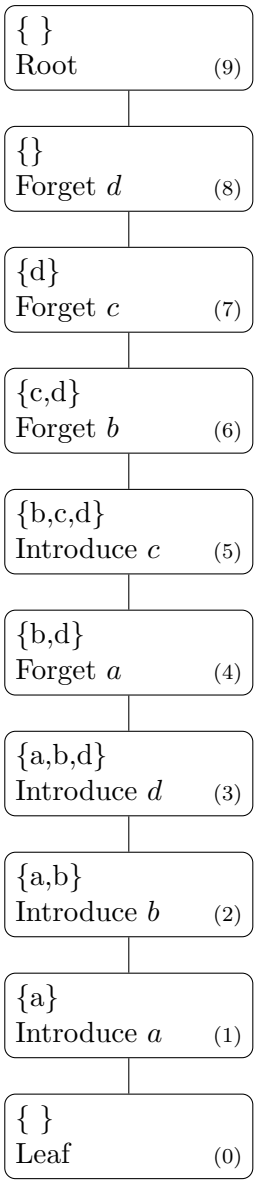

Figure 14: Nice Tree Decomposition of the Rhombus Graph

A.2 Simulation of Table Population of graph one

The table $c[\mathrm{i}, I, \mathcal{F}]$ represents the population of the values as we move through the tree decomposition steps. Below, we provide a simulation of this process.

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
0	$\emptyset$	$\emptyset$	0
1	a	$\emptyset$	1
1	$\emptyset$	$\emptyset$	0
2	$\{a, b\}$	$\emptyset$	4
2	$\{a\}$	$\emptyset$	1
2	$\{b\}$	$\emptyset$	3
2	$\emptyset$	$\emptyset$	0
3	$\{a, b, d\}$	$\emptyset$	6
3	$\{a, b\}$	$\emptyset$	4
3	$\{a, d\}$	$\emptyset$	3
3	$\{b, d\}$	$\emptyset$	5
3	$\{a\}$	$\emptyset$	1
3	$\{b\}$	$\emptyset$	3
3	$\{d\}$	$\emptyset$	2
3	$\emptyset$	$\emptyset$	0
4	$\{b, d\}$	$\emptyset$	6
4	$\{b, d\}$	$\{b, d\}$	5
4	$\{b\}$	$\emptyset$	4
4	$\{b\}$	$\{b, d\}$	3
4	$\{d\}$	$\emptyset$	3
4	$\{d\}$	$\{b, d\}$	2
4	$\emptyset$	$\emptyset$	1
4	$\emptyset$	$\{b, d\}$	0
5	$\{b, c, d\}$	$\emptyset$	10
5	$\{b, c, d\}$	$\{b, d\}$	9
5	$\{b, d\}$	$\emptyset$	6
5	$\{b, d\}$	$\{b, d\}$	5

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
5	$\{b, c\}$	$\emptyset$	8
5	$\{b, c\}$	$\{b, d\}$	7
5	$\{b\}$	$\emptyset$	4
5	$\{b\}$	$\{b, d\}$	3
5	$\{c, d\}$	$\emptyset$	7
5	$\{c, d\}$	$\{b, d\}$	6
5	$\{d\}$	$\emptyset$	3
5	$\{d\}$	$\{b, d\}$	2
5	$\{c\}$	$\emptyset$	5
5	$\{c\}$	$\{b, d\}$	4
5	$\emptyset$	$\emptyset$	1
5	$\emptyset$	$\{b, d\}$	∞
6	$\{c, d\}$	$\emptyset$	10
6	$\{c, d\}$	$\emptyset$	9
6	$\{d\}$	$\emptyset$	6
6	$\{d\}$	$\emptyset$	5
6	$\{c\}$	$\emptyset$	8
6	$\{c\}$	$\emptyset$	7
6	$\emptyset$	$\emptyset$	4
6	$\emptyset$	$\emptyset$	3
6	$\{c, d\}$	$\emptyset$	7
6	$\{c, d\}$	$\emptyset$	6
6	$\{d\}$	$\emptyset$	3
6	$\{d\}$	$\emptyset$	2
6	$\{c\}$	$\emptyset$	5
6	$\{c\}$	$\emptyset$	4
6	$\emptyset$	$\emptyset$	1
6	$\emptyset$	$\emptyset$	∞

Notice that at this point, at index 6, we are taking minimum values that we can resolve. That is, some values can be thrown out, because they are suboptimal.

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
6	$\{c, d\}$	$\emptyset$	6
6	$\{d\}$	$\emptyset$	2
6	$\{c\}$	$\emptyset$	4
6	$\emptyset$	$\emptyset$	1

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
7	$\{d\}$	$\emptyset$	2
7	$\emptyset$	$\emptyset$	1
8	$\emptyset$	$\emptyset$	1
9	$\emptyset$	$\emptyset$	1

There is one value at the root and it is the minimum weight of a set F that breaks every 4-cycle in a planar graph G.

A.3 Example two graph and Nice Tree Decomposition

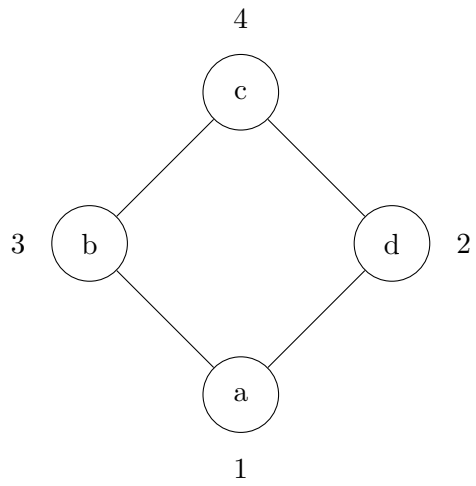

Figure 15: Weighted Rhombus Graph G

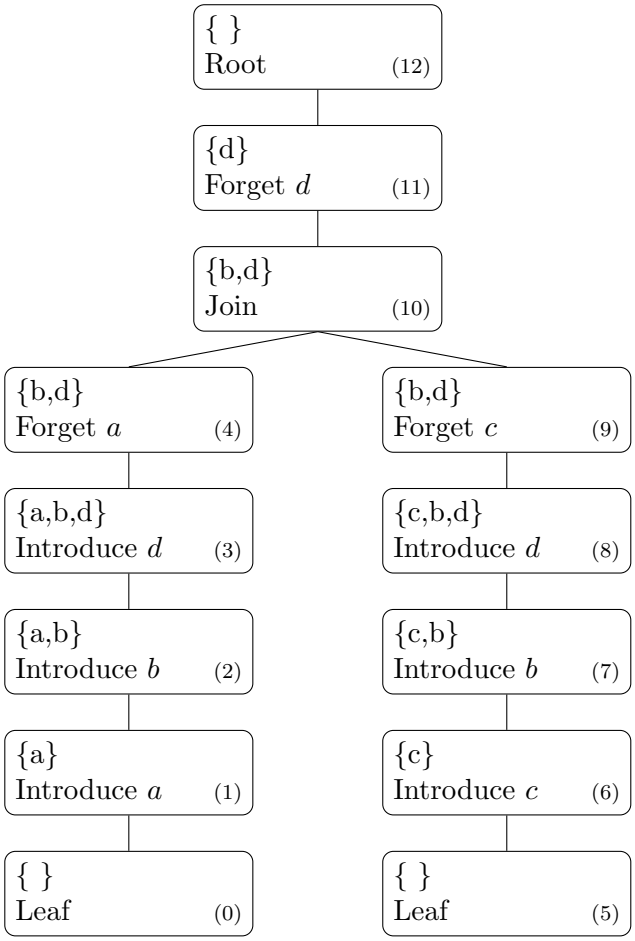

Figure 16: Alternative Tree Decomposition with Join Node at {b,d}

A.4 Simulation of Table Population of graph two

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
0	$\emptyset$	$\emptyset$	**0**
1	$\{a\}$	$\emptyset$	**1**
1	$\emptyset$	$\emptyset$	**0**
2	$\{a, b\}$	$\emptyset$	**4**
2	$\{a\}$	$\emptyset$	**1**
2	$\{b\}$	$\emptyset$	**3**
2	$\emptyset$	$\emptyset$	**0**
3	$\{a, b, d\}$	$\emptyset$	**6**
3	$\{a, b\}$	$\emptyset$	**4**
3	$\{a, d\}$	$\emptyset$	**3**
3	$\{b, d\}$	$\emptyset$	**5**
3	$\{a\}$	$\emptyset$	**1**
3	$\{b\}$	$\emptyset$	**3**
3	$\{d\}$	$\emptyset$	**2**
3	$\emptyset$	$\emptyset$	**0**
4	$\{b, d\}$	$\emptyset$	**6**
4	$\{b, d\}$	$\{b, d\}$	**5**
4	$\{b\}$	$\emptyset$	**4**
4	$\{b\}$	$\{b, d\}$	**3**
4	$\{d\}$	$\emptyset$	**3**
4	$\{d\}$	$\{b, d\}$	**2**
4	$\emptyset$	$\emptyset$	**1**
4	$\emptyset$	$\{b, d\}$	**0**

Index i	**Set** I	**Flag** $\mathcal{F}$	**Value** $c[i, I, \mathcal{F}]$
5	$\emptyset$	$\emptyset$	**0**
6	$\{c\}$	$\emptyset$	**4**
6	$\emptyset$	$\emptyset$	**0**
7	$\{c, b\}$	$\emptyset$	**7**
7	$\{c\}$	$\emptyset$	**4**
7	$\{b\}$	$\emptyset$	**3**
7	$\emptyset$	$\emptyset$	**0**
8	$\{c, b, d\}$	$\emptyset$	**9**
8	$\{c, b\}$	$\emptyset$	**7**
8	$\{c, d\}$	$\emptyset$	**6**
8	$\{b, d\}$	$\emptyset$	**5**
8	$\{c\}$	$\emptyset$	**4**
8	$\{b\}$	$\emptyset$	**3**
8	$\{d\}$	$\emptyset$	**2**
8	$\emptyset$	$\emptyset$	**0**
9	$\{b, d\}$	$\emptyset$	**9**
9	$\{b, d\}$	$\{b, d\}$	**5**
9	$\{b\}$	$\emptyset$	**7**
9	$\{b\}$	$\{b, d\}$	**3**
9	$\{d\}$	$\emptyset$	**6**
9	$\{d\}$	$\{b, d\}$	**2**
9	$\emptyset$	$\emptyset$	**4**
9	$\emptyset$	$\{b, d\}$	**0**